第0章: はじめに — 書籍管理アプリを作ろう！

> **「プログラミングを学ぶ最良の方法は、実際にものを作ることです。」**
>
> このチュートリアルでは、モダン（modern：現代的、最新の、という意味の英単語）なWeb技術を使って、実用的な**書籍管理アプリ**をゼロから構築します。初心者の方でも安心して進められるように、一つひとつ丁寧に解説していきます。
>
> なお、文章の中の バッククォート（\ ）で囲まれた部分` は「コード（プログラムの一部）」を表す印です。プログラミング用語や、コマンド・ファイル名などをこの記号で囲って強調しています。

この章のゴールと地図

この第0章は、**コードを本格的に書き始める前の「準備運動」**の章です。料理にたとえると、包丁を握る前に「どんな料理を作るのか（完成写真）」「台所のどこに何があるのか（道具の場所）」「どんな順番で作るのか（レシピの流れ）」を確かめる段階にあたります。ここで全体像をつかんでおくと、第1章以降で迷子になりにくくなります。

この章を読み終えると、こうなります（ゴール）

- [ ]「ブラウザ」「サーバー」「URL」「ポート」「ターミナル」などの基本用語を、自分の言葉で説明できる
- [ ] 書籍管理アプリが **ブラウザ** ⇄ **Next.js** ⇄ **Supabase** のやり取りで動いていることを、図を使って説明できる
- [ ] `node ファイル名` で小さなプログラムを動かし、エラーが出たときに「どこを読めばいいか」が分かる
- [ ] 第1章〜第11章の道のりと、TypeScript・React・Next.js・Supabase それぞれの役割分担が分かる

この章の地図

この章は、おおまかに3つのパートに分かれています。①は辞書のように何度も戻ってくる「言葉と道具」、②はアプリの「全体像」、③は「学習の進め方」と理解度の確認です。

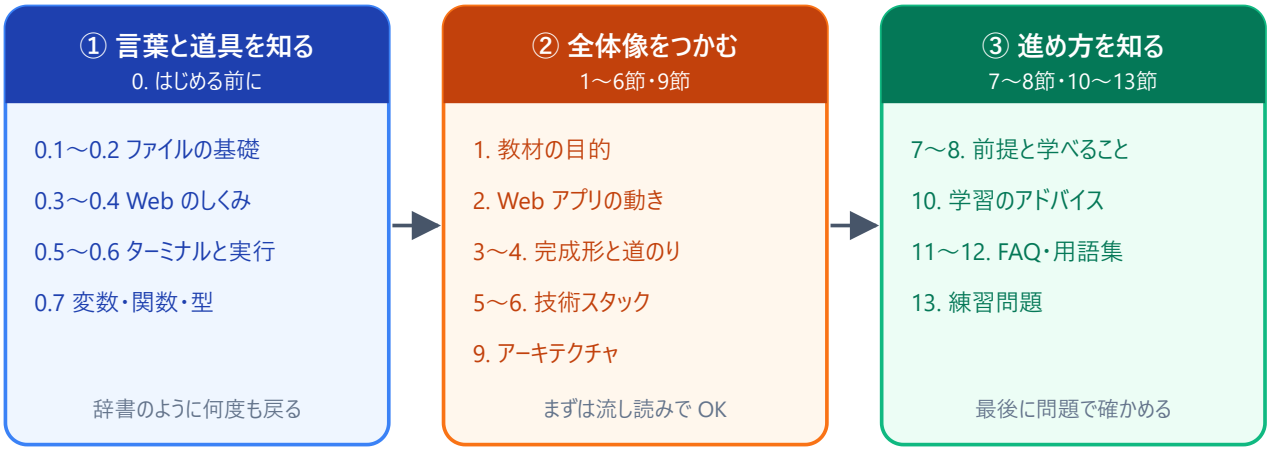


図0-1: この章の地図。①で言葉に慣れ、②でアプリの全体像をつかみ、③で進め方を知って理解度を確かめます

> **ヒント:** この章には、`node hello.js` のように**実際に動かして試せる小さなプログラム**がいくつか出てきます。まだ Node.js（第1章でインストールします）が入っていなければ、**今は読むだけで大丈夫**です。第1章で環境を整えたあとに戻ってきて、実際に動かしてみてください。「▼ 実行結果」には、筆者が実際に動かしたときの出力をそのまま載せています（パソコンのユーザー名が入る部分だけ `taro` に置き換えています）。

この教材の読み方（囲みや記号の意味）

見た目	意味
▼ このコードがやること（先に日本語で）	コードを読む前に、何をするコードなのかを日本語で先に説明しています
▼ 実行するコマンド / ▼ 実行結果	ターミナルに打つ命令と、そのとき実際に画面に出る文字です
解説1・解説2…	コードをいくつかの塊に分けて、1つずつ説明しています
つまずきポイント	初心者がよくハマる所です。**NG（まちがった例）** → **OK（正しい例）** の順に並べています
ヒント / 用語 / 注意	ちょっとした補足 / 言葉の意味 / 気をつけること です
図0-N	図の番号です。「図0-3」は「第0章の3枚目の図」という意味です

0. はじめる前に ― 「そもそも何？」を全部解説

この章を読み始める前に、まず**プログラミング・Web開発の最も基礎的な用語**を解説します。「もう知ってるよ」という方は読み飛ばして「1. この教材の目的と対象読者」から始めてください。逆に「これから初めてプログラミングをやります」という方は、ここを丁寧に読むと後の章が驚くほどスムーズに進みます。

> **記号の読み方の予備知識:** プログラミングでは見慣れない記号がたくさん出てきます。よく出るものを先にメモしておきましょう。- `"..."` ダブルクォート（double quote、二重引用符）。文字列を囲む記号。- `'...'` シングルクォート（single quote、一重引用符）。同じく文字列を囲む記号。- `` `...` `` バッククォート（back quote）／テンプレートリテラル。文字列を囲みつつ `${変数}` の埋め込みもできる。- `;` セミコロン（semicolon）。「文（命令）の終わり」を表す印。- `,` カンマ（comma）。値や項目の区切り。- `{ }` 中カッコ（波カッコ、curly braces）。コードの「ブロック」やオブジェクトを囲む。- `[ ]` 角カッコ（square brackets）。配列（リスト）を囲む。- `( )` 丸カッコ（parentheses）。関数の引数や、計算の優先順位に使う。- `//` 行コメント。`//` から行末までは説明文として扱われ、プログラムとしては実行されない。- `/* ... */` ブロックコメント。複数行をまとめてコメントにする。

0.1 プログラミングとは何か

プログラミング（programming） とは、コンピュータに「やってほしいこと」を**手順書（プログラム）** として書く作業です。コンピュータは指示された通りにしか動かないので、何をしてほしいかを「コンピュータが理解できる言葉（プログラミング言語）」で書く必要があります。

身近な例で言うと、料理のレシピや、家具の組立説明書に近いものです。「(1)小麦粉を200g計る → (2)ボウルに入れる → (3)水を加える」のように、**順番と条件**を厳密に書くことで、誰がやっても（コンピュータが何度実行しても）同じ結果になるようにします。

> **覚えておきたい3要素:** - **プログラム（program）**：コンピュータへの指示書そのもの。「ソースコード（source code、元になるコードという意味）」「コード」とも言う。- **プログラミング言語（programming language）**：その指示書を書くための言語。日本語/英語のように種類があり、本書では **TypeScript（タイプスクリプト）** を使う。- **プログラマー（programmer）/エンジニア（engineer）**：プログラムを書く人。

0.2 ソースコード・ファイル・拡張子の超基礎

プログラムは **テキストファイル**（text file：文字だけが入っているファイル）に書きます。Word文書のような装飾（太字・色など）を持たない、純粋な文字だけのファイルです。ファイルの末尾には **拡張子（かくちょうし、extension）** という「.（ドット）＋数文字」が付きます。拡張子を見れば「中身がどんな種類のファイルか」が分かるようになっています。

拡張子	何のファイルか	このチュートリアルでの登場場面
`.html`	Webページの構造を書くファイル	結果として生成されるが、自分ではあまり書かない
`.css`	見た目（色・サイズ・配置）を指定するファイル	第9章のスタイリングで登場
`.js`	JavaScript（ブラウザで動くプログラミング言語）を書くファイル	あまり直接書かない
`.ts`	TypeScript（JavaScriptに型を足した言語）を書くファイル	第2章以降ずっと使う
`.tsx`	TypeScript＋JSX（HTMLっぽい記法）を書くファイル	Reactのコンポーネントで使う
`.json`	設定情報やデータをやり取りする形式のファイル	`package.json` などで頻出
`.md`	Markdown（このチュートリアル自身の形式）。説明文用	教材ファイルの拡張子

▼ **例:** `App.tsx` という名前のファイルは、「中身が TypeScript＋JSX で書かれた、`App` という名前のファイル」を意味します。ファイル名（`App`）と拡張子（`.tsx`）は「.」（ドット）で区切られています。慣習的にコンポーネントを書くファイルは先頭を大文字（`App`）にすることが多いです。



図0-2: 左はファイル名の部品。右は、拡張子が隠れているせいで本当の名前が分からなくなる、Windows でよくある落とし穴

> **つまずきポイント:**「`hello.js` を作ったはずなのに、`node hello.js` を実行すると `Cannot find module`（モジュール＝ファイルが見つからない）と言われる」——Windows でとても多いトラブルです。メモ帳などで保存すると、気づかないうちに本当の名前が `hello.js.txt` になっていることがあります。エクスプローラーの初期設定では拡張子が隠れているので、見た目は `hello.js` のままなのが厄介なところです。

NG: 本当の名前が `hello.js.txt` になっている（`ls` でフォルダの中身を確かめると分かる）

```
PS C:\Users\taro\study> ls

    ディレクトリ: C:\Users\taro\study


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        2026/10/15     20:30             48 hello.js.txt

PS C:\Users\taro\study> node hello.js
node:internal/modules/cjs/loader:1478
  throw err;
  ^

Error: Cannot find module 'C:\Users\taro\study\hello.js'
（このあとに at で始まる行が続きますが、省略します）
```

OK: 名前を `hello.js` に直してから実行する

```
# Rename-Item（名前の変更）で、hello.js.txt を hello.js に変える
Rename-Item hello.js.txt hello.js
# もう一度実行する
node hello.js
```

```
こんにちは、世界！
```

- 再発を防ぐには、拡張子をいつも表示する設定にしておきましょう。Windows 11 ならエクスプローラーの「表示」→「表示」→「ファイル名拡張子」にチェック、Mac なら Finder の「設定」→「詳細」→「すべてのファイル名拡張子を表示」にチェックを入れます。
- そもそもメモ帳ではなく **VS Code**（第1章で入れます）でファイルを作れば、この問題はほとんど起きません。

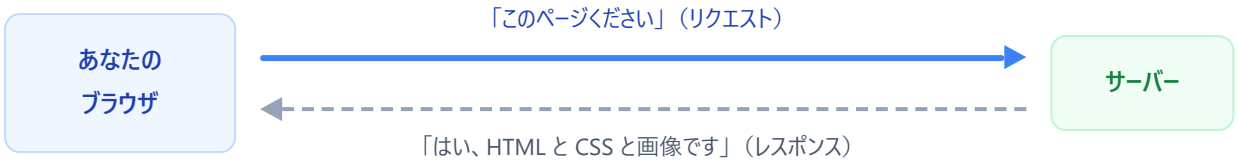
0.3 Webブラウザとサーバーの関係

あなたが今このページを見ているソフトを **ブラウザ**（**browser、ブラウズ＝閲覧するもの**） と呼びます。Chrome（クローム）、Edge（エッジ）、Safari（サファリ）、Firefox（ファイヤーフォックス）などです。

ブラウザの仕事は大きく2つあります。

1. **インターネット上のどこかにあるサーバーから、HTML・CSS・JavaScript などのファイルを取ってくる**
2. **取ってきたファイルを解釈して、画面に表示する／プログラムを実行する**

サーバー（**server、サーブ＝提供する者**） とは、24時間ずっと電源が入っていて「リクエストが来たら何かを返す」役割のコンピュータのことです。Amazonの倉庫の係員のように「注文（リクエスト）」を受けて「商品（レスポンス）」を返します。



上の矢印は「お願い（リクエスト）の向き」、下の矢印は「サーバーが応答（レスポンス）を返す向き」。矢印の上のラベルでやり取りの種類を示しています。

▼ **実例:** ブラウザのアドレスバーに `https://example.com/about` と打つと、

1. ブラウザは `example.com` というサーバーを探しに行く（DNS という仕組みで住所を解決。DNS = Domain Name System、ドメイン名をIPアドレスに変換する仕組み）
2. そのサーバーに「`/about` のページが欲しい」とリクエストする（`/about` の部分を「パス」と呼ぶ。サイト内のどこのページかを示す）
3. サーバーは HTML を返す
4. ブラウザはその HTML を読み解いて画面を描画する

この流れを、登場人物ごとの縦線を使って図にすると次のようになります（上から下へ時間が進みます）。DNS サーバーは「インターネットの住所録」で、`example.com` のような名前を、コンピュータ同士が使う番地（IPアドレス）に変換してくれます。

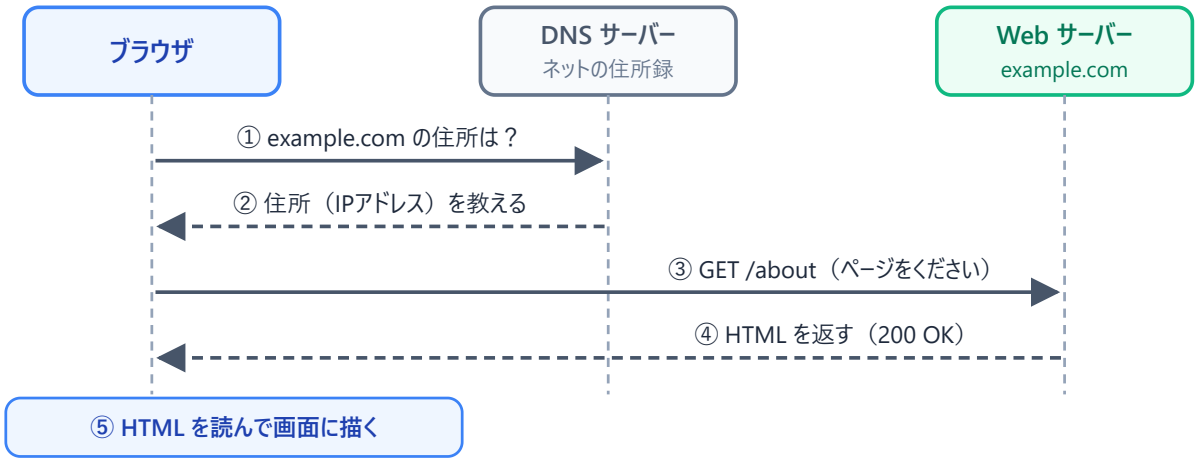


図0-3: URL を入力してから画面が出るまで。実線はお願い（リクエスト）、点線は返事（レスポンス）。①②で住所を調べ、③④でページを受け取り、⑤で画面に描きます

> **用語: IPアドレス（アイピーアドレス）** … インターネットにつながった機械1台1台に付いている番地です。`192.0.2.10` のような数字の並びで、人間には覚えにくいので、`example.com` のような名前（ドメイン名）と DNS を組み合わせて使います。

このやり取りを **HTTP（HyperText Transfer Protocol、ハイパーテキスト転送規約）通信** と呼びます。`https://` の `s` は Secure（セキュア、暗号化されている）の意味です。URL（ユーアールエル、Uniform Resource Locator）の構

造は次のようになっています。

```
https :// example.com : 443 / about ? q=book #section1
  |        |           |     |       |        |
  |        |           |     |       |        └── フラグメント（ページ内の位置。先頭が #）
  |        |           |     |       └── クエリ文字列（追加情報。先頭が ?）
  |        |           |     └── パス（サイト内のどのページか）
  |        |           └── ポート番号（省略可。https の既定値は 443）
  |        └── ホスト名／ドメイン（どのサーバーか）
  └── スキーム（通信の種類。http や https）
```

URL をプログラムで部品に分けてみる

> **▼ このコードがやること（先に日本語で）**：上の図で見た URL の「部品」を、プログラムに分解させて1つずつ表示します。JavaScript / TypeScript には `URL` という「URL を部品に分けてくれる道具」が最初から入っているので、それを使います。本書で何度も開く `http://localhost:3000/...` という URL も一緒に分解してみます。

```ts
// 分解してみたい URL を2つ、配列に入れて用意する
const addresses: string[] = [
  "https://example.com:443/about?q=book#section1",
  "http://localhost:3000/books/new",
];

// 配列の中身を1つずつ取り出して、同じ処理をくり返す
for (const address of addresses) {
  // new URL(...) で、URL の文字列を「部品ごとに分けた形」に変換する
  const url = new URL(address);

  console.log(`▼ ${address}`);
  console.log(`  スキーム     : ${url.protocol}`);
  console.log(`  ホスト名     : ${url.hostname}`);
  console.log(`  ポート番号   : "${url.port}"`);
  console.log(`  パス         : ${url.pathname}`);
  console.log(`  クエリ文字列: ${url.search}`);
  console.log(`  q の値       : ${url.searchParams.get("q")}`);
  console.log(`  フラグメント: ${url.hash}`);
}
```

▼ 実行するコマンド（`url.ts` という名前で保存したフォルダで）

```
node url.ts
```

▼ 実行結果

```
▼ https://example.com:443/about?q=book#section1
  スキーム     : https:
  ホスト名     : example.com
  ポート番号   : ""
  パス         : /about
  クエリ文字列: ?q=book
  q の値       : book
  フラグメント: #section1
▼ http://localhost:3000/books/new
  スキーム     : http:
  ホスト名     : localhost
  ポート番号   : "3000"
  パス         : /books/new
  クエリ文字列:
  q の値       : null
  フラグメント:
```

▼ コードを1つずつ分解して解説

解説1: URL の文字列を配列に入れておく

```
const addresses: string[] = [
  "https://example.com:443/about?q=book#section1",
  "http://localhost:3000/books/new",
];
```

- `const addresses` は、「addresses（アドレスたち）という名前の箱」を作る宣言です。
- `: string[]` は型注釈で、「文字列（string）が並んだ配列（`[]`）だけを入れてよい」というルールです。
- `[ ... ]` の中に、カンマ区切りで2つの URL を並べています。

解説2: for...of で1つずつ取り出す

```
for (const address of addresses) {
```

- 「addresses の中身を先頭から1つずつ取り出して `address` に入れ、`{ }` の中を実行する」をくり返す書き方です。URL が2つあるので、2回くり返されます。

解説3: new URL で分解して、部品を表示する

```
const url = new URL(address);
console.log(`  ポート番号   : "${url.port}"`);
```

- `new URL(文字列)` で、URL を「部品ごとに取り出せる形（オブジェクト）」に変換します。

- `url.protocol`（スキーム）、`url.hostname`（ホスト名）、`url.port`（ポート番号）、`url.pathname`（パス）、`url.search`（クエリ文字列）、`url.hash`（フラグメント）のように、`.部品名`で取り出せます。
- `url.searchParams.get("q")`は、クエリ文字列の中から`q=`の値だけを取り出します。2つ目のURLには`q`が無いので、「値が無い」ことを表す`null`が表示されました。
- `` `...${式}...` ``はテンプレートリテラルといい、文字列の中に値を埋め込めます。ポート番号は空っぽでも見えるように`"`で囲んで表示しています。

> **つまずきポイント:** 1つ目のURLには`:443`と書いたのに、ポート番号が`""`（空っぽ）と表示されました。これは**httpsの既定（デフォルト）のポート番号が443なので、書いても書かなくても同じ意味になり、「省略された」扱いになる**からです（httpの既定は80）。逆に、開発中に使う`localhost:3000`の`3000`は既定値ではないので**省略できません。**
>
> - NG: `http://localhost/books/new` … ポート番号を省略したので80番の窓口に行ってしまい、開発中のアプリにつながらない
> - OK: `http://localhost:3000/books/new` … `npm run dev`が待っている3000番の窓口に届く

0.4 フロントエンドとバックエンドの違い

Web開発の現場ではよく「**フロントエンド／バックエンド／フルスタック**」という言葉を使います。「フロント（front）」は前、「バック（back）」は後ろ、「エンド（end）」は端という意味で、ユーザーから見える「前側」とサーバーで動いている「後ろ側」を表しています。

用語	意味	担当する技術（本書で使うもの）
フロントエンド（frontend）	画面側、ユーザーが直接操作する部分	HTML / CSS / TypeScript / React / Next.js
バックエンド（backend）	サーバー側、データの保存や認証などを担当	Supabase（自動でAPIを生成）
データベース（database）	データを保存しておく場所	PostgreSQL（Supabase内蔵）
フルスタック（fullstack）	フロントもバックも両方できる人/構成	本書はこの構成を目指す

> **本書のポイント:** 通常はバックエンドを自分でゼロから作る必要がありますが、**Supabase（スーパベース）**がそれを肩代わりしてくれます。だから「フロントエンドだけに集中していれば、結果としてフルスタックなアプリが完成する」というお得な構成になっています。

0.5 コードを書く・動かすための道具

このチュートリアルで実際に使う道具（ツール）の名前を、先に頭出ししておきます。詳しいインストール方法は第1章で説明します。

道具	役割	例えるなら
VS Code	コードを書くためのエディタ（高機能なメモ帳）	原稿用紙＋校閲機能
ターミナル（コマンドプロンプト/PowerShell/Terminal）	キーボードから命令文を打ってPCを操作する画面	レストランでの口頭注文
Node.js	JavaScript/TypeScript を**ブラウザ外**でも動かせるようにする実行環境	家庭用キッチン
npm（エヌピーエム）	他人が作った便利なプログラム部品（パッケージ）をダウンロードして管理するツール	アマゾン的な部品通販
Git（ギット）	「いつ・誰が・何を変えたか」を記録するバージョン管理ツール	編集履歴付きノート
GitHub（ギットハブ）	Gitで管理したコードをインターネット上に保管する場所	コード版のクラウドストレージ
ブラウザ	作ったWebアプリを表示・確認するソフト	完成品の試食皿

> **ターミナル画面の「プロンプト記号」について:** ターミナルを開くと、命令を待ち受ける目印（プロンプト、prompt）として `$` 、`>` 、`%` 、`#` などが行頭に出ます。これは「ここから先に命令を打ってね」というだけのマークで、命令の一部ではありません。本書のサンプルでも `$ npm install` のように書かれていたら、`$` は打ち込まずに `npm install` の部分だけを入力します。- `$` … macOS/Linux の Bash や Zsh で標準的 - `>` … Windows のコマンドプロンプトや PowerShell でよく見る - `%` … macOS の Zsh の既定 - `#` … 管理者（root）として開いた時の印

実際のターミナルは、次のような見た目をしています（Windows の PowerShell の例）。黒い画面に文字だけが並ぶので最初は戸惑いますが、「**プロンプト → 自分で打つコマンド → 結果の出力 → また次のプロンプト**」のくり返しになっていることが分かれば怖くありません。

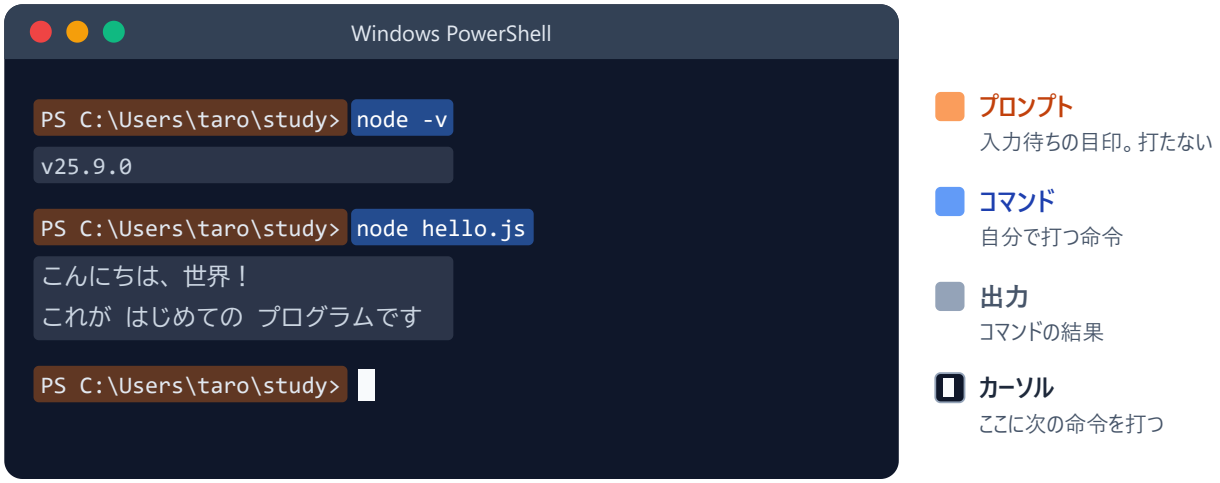


図0-4: ターミナル（PowerShell）の画面イメージ。橙がプロンプト、青が自分で打つコマンド、灰色が出力です。バージョン番号やフォルダ名は人によって違います

> **つまずきポイント:** プロンプト記号の `$` や `PS C:\...>` まで一緒に打ち込んでしまう。本や Web の記事からコマンドをコピーしたときによく起きます。

NG: `$` まで打ってしまった（PowerShell の場合）

```
PS C:\Users\taro\study> $ node -v
$ : The term '$' is not recognized as the name of a cmdlet, function, script file, or
operable program. Check the spelling of the name, or if a path was included, verify
that the path is correct and try again.
（このあとに、まちがえた位置を示す行が続きます）
```

OK: `$` の右側（`node -v`）だけを打つ

```
PS C:\Users\taro\study> node -v
v25.9.0
```

- 英語のメッセージは「`$` という名前のコマンドは見つかりません。つづりを確かめてください」という意味です。日本語の環境では「用語 '$' は、コマンドレット、関数、スクリプト ファイル、または操作可能なプログラムの名前として認識されません」のように表示されます。
- `v25.9.0` は Node.js のバージョン番号です。インストールした時期によって数字は違います。

> **つまずきポイント:** 日本語入力（IME）がオンのままコマンドを打つと、`node` が全角の `ｎｏｄｅ` になってしまいます。見た目がほとんど同じなので、気づきにくいミスです。

NG: 全角で `ｎｏｄｅ -v` と打ってしまった

```
PS C:\Users\taro\study> ｎｏｄｅ -v
ｎｏｄｅ : The term 'ｎｏｄｅ' is not recognized as the name of a cmdlet, function,
script file, or operable program. Check the spelling of the name, or if a path was
included, verify that the path is correct and try again.
（このあとに、まちがえた位置を示す行が続きます）
```

OK: 日本語入力をオフ（半角英数）にしてから打つ

```
PS C:\Users\taro\study> node -v
v25.9.0
```

- コマンドやコードの記号は **半角英数字** で打つのが大原則です。Windows ではキーボードの「半角/全角」キー、Mac では「英数」キーで切り替えます。
- 区切りのスペースも半角にします。全角スペースは、ターミナルの種類によって通ったりエラーになったりします（例: Mac や Linux の bash では `node　hello.js: command not found` となり、`node　hello.js` 全体が1つのコマンド名だと思われてしまいます）。

0.6 「コードを実行する」ってどういうこと？

このチュートリアルでは何度も「**このコードを実行してみましょう**」と書きます。「実行」とは、書いたプログラムをコンピュータに**読み取らせて、その通りに動かす**ことです。本書では2通りの実行方法が登場します。

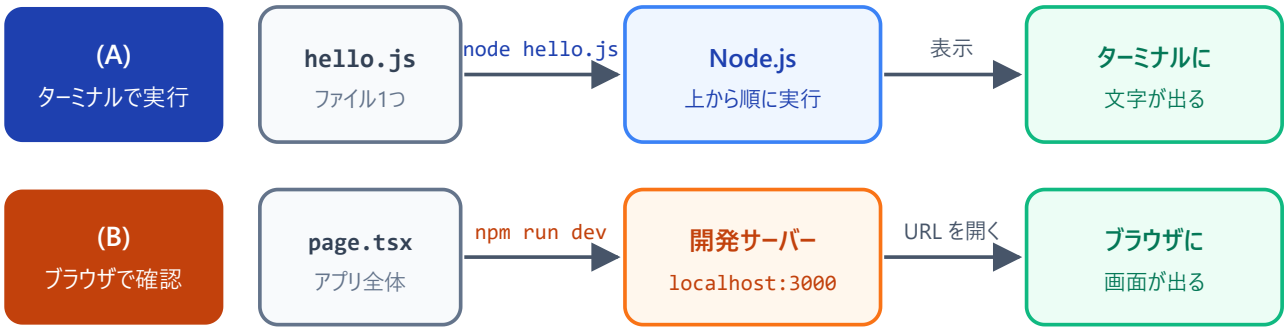


図0-5: 本書に出てくる2通りの「実行」。(A) は1つのファイルをその場で動かす方法、(B) はアプリ全体を動かしてブラウザで確かめる方法です

(A) ターミナルでコマンドを打って実行する

ターミナルを開いて、たとえば次のように入力します。

```
node hello.js
# ↑ node というコマンドで、hello.js というファイルを実行する命令
# node              : Node.js を起動するコマンド名（実行ファイル）
# （半角スペース）  : コマンドと引数の区切り。スペースは必ず半角（全角だとエラー）
# hello.js          : 実行したいファイル名（拡張子も含めて指定する）
# Enter を押すとこの行が実行され、Node.js が hello.js を読み込んで処理する
```

これは「`hello.js` というファイルの中身を、Node.js（ノードジェイエス）に読ませて実行してね」という意味です。Enterキーを押すと、画面（ターミナル）に結果が出力されます。

▼ 実行結果の例:

```
こんにちは、世界！
```

> 「**コマンド」とは何？**: コマンド（command）= 命令。ターミナルに打つ「短い英単語の指示」のこと。`node`、`npm install`、`git status` などはすべてコマンドです。コマンドの後ろにスペースを空けて足す追加情報を「引数（ひきすう、argument）」、ハイフンが付いた指定（`-r` や `--save-dev` など）を「オプション／フラグ（flag）」と呼びます。

はじめてのプログラム hello.js を作って動かす

上で `node hello.js` を実行しましたが、`hello.js` の中身はまだ見せていませんでした。中身はたったこれだけです。

> **▼ このコードがやること（先に日本語で）**：ターミナルに2行の文章を表示します。プログラムは**上から下へ1行ずつ順番に**実行される、ということを確かめるための最小のプログラムです。

```
// 画面（ターミナル）に「こんにちは、世界！」と表示する
console.log("こんにちは、世界！");

// 2行目も表示してみる（上から順番に実行される）
console.log("これが はじめての プログラムです");
```

▼ 実行するコマンド（`hello.js` を保存したフォルダで）

```
node hello.js
```

▼ 実行結果

```
こんにちは、世界！
これが はじめての プログラムです
```

▼ コードを1つずつ分解して解説

解説1: `//` で始まる行はコメント

```
// 画面（ターミナル）に「こんにちは、世界！」と表示する
```

- `//` から行の終わりまでは「人間向けのメモ（コメント）」で、コンピュータは読み飛ばします。
- 本書のサンプルでは、**コメントは説明したいコードの1行上**に書いています。

解説2: `console.log(...)` で表示する

```
console.log("こんにちは、世界！");
```

- `console.log( )` は「カッコの中身をコンソール（ターミナルやブラウザの出力欄）に表示する」命令です。
- `"こんにちは、世界！"` のように `"` で囲んだものは**文字列**（文字の並び）です。
- 行の最後の `;`（セミコロン）は「この命令はここまで」という区切りです。

解説3: 上から順番に実行される

```
console.log("こんにちは、世界！");
console.log("これが はじめての プログラムです");
```

- 1つ目の `console.log` が先に、2つ目があとに実行されるので、結果も同じ順番で2行出ます。2行の順番を入れ替えて保存し、もう一度 `node hello.js` を実行すると、表示の順番も入れ替わります。試してみましょう。

> **つまずきポイント:** ファイルがあるのとは**違うフォルダ**で `node hello.js` を実行してしまう。ターミナルは「今いるフォルダ（カレントディレクトリ）」の中でファイルを探すので、場所がずれていると見つかりません。

NG: `hello.js` が無いフォルダ（ここでは `C:\Users\taro`）で実行した

```
PS C:\Users\taro> node hello.js
node:internal/modules/cjs/loader:1478
  throw err;
  ^

Error: Cannot find module 'C:\Users\taro\hello.js'
（このあとに at で始まる行が続きますが、省略します）
```

OK: `cd` で `hello.js` のあるフォルダに移動してから実行する

```
# cd（change directory）で、hello.js がある study フォルダに移動する
cd study
# 移動した先で実行する
node hello.js
```

```
こんにちは、世界！
これが はじめての プログラムです
```

- `Cannot find module '...'` の `'...'` の部分には「Node.js が探しに行った場所」が書かれています。そこに本当にファイルがあるか、エクスプローラーや `ls` で確かめましょう。
- プロンプトの `PS C:\Users\taro>` の部分には「今いるフォルダ」が表示されています。コマンドを打つ前にここを見るクセをつけると、このミスはぐっと減ります。

(B) ブラウザで開いて実行する

Webアプリの場合、コードはサーバーが動かしたり、ブラウザが動かしたりします。本書では `npm run dev` というコマンドで**開発用のサーバー**を起動し、ブラウザから `http://localhost:3000` にアクセスして動作を確認します。

▼ 起動コマンドと出力の例:

```
npm run dev
# npm         : Node.js に付属するパッケージマネージャ（部品管理ツール）のコマンド
# run         :  「スクリプト（あらかじめ登録された命令）を実行する」サブコマンド
# dev         : 実行するスクリプト名。慣習的に「開発用サーバーを起動する」用途で使われる
# まとめると「package.json に書かれている dev スクリプトを動かしてね」という意味
```

```
> book-management@0.1.0 dev
> next dev

▲ Next.js 16.4.1 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://192.168.x.x:3000
- Environments: .env.local
✓ Ready in 7.0s
```

（第6章で作るプロジェクトを、Next.js 16.4.1 で実際に起動したときの出力です。このあとにも数行続きます。`Network:` の数字や `Ready in` の秒数は環境によって変わり、`.env.local`（第6章で作る設定ファイル）が無ければ `Environments:` の行は出ません）

ブラウザで `http://localhost:3000` を開くと、自分で作ったページが表示されます。`localhost`（ローカルホスト）は「自分自身のPC」を意味する特別な住所、`3000` はポート番号（同じPC内のどの窓口かの番号）です。`http://`

と `localhost` の間にある `://` はURLの「スキーム（通信の種類）」と「住所」を区切る決まりの記号です。

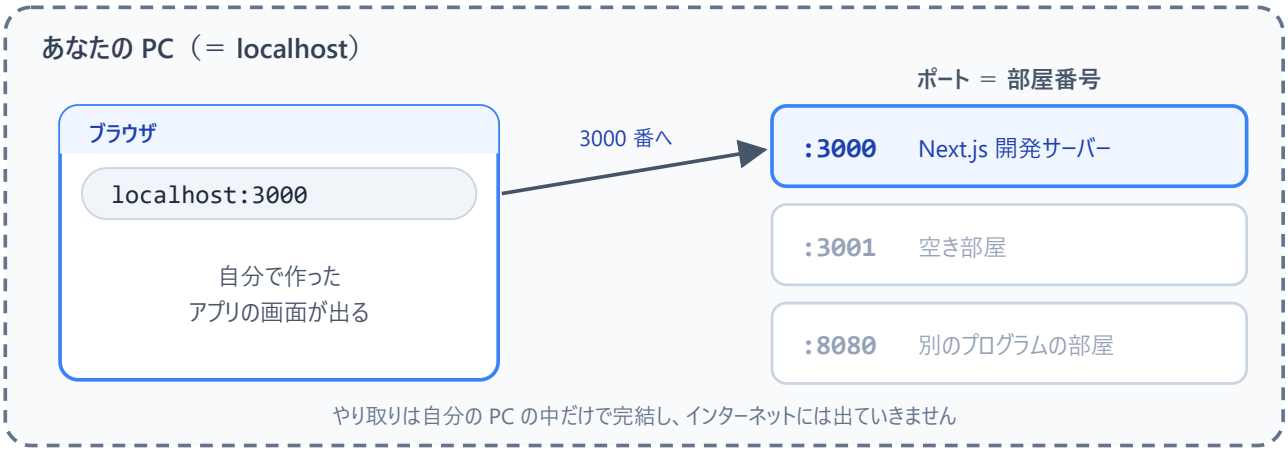


図0-6: localhost とポート番号。localhost が建物の住所、:3000 が部屋番号です。npm run dev で起動した開発サーバーが 3000 番の部屋で待っています

> **用語: ポート（port）** … 1台のコンピュータの中にある「窓口の番号」です。マンションにたとえると、`localhost` が建物の住所、`3000` が部屋番号にあたります。1つの部屋（ポート）には、同時に1つのプログラムしか入れません。

コードを書いてから、画面に出るまで

`npm run dev` で起動した開発サーバーは、**ファイルの変更を見張っています**。そのため、コードを書き換えて保存するだけで、ブラウザの画面が自動で新しくなります。第4章以降、この流れを何百回もくり返すことになります。

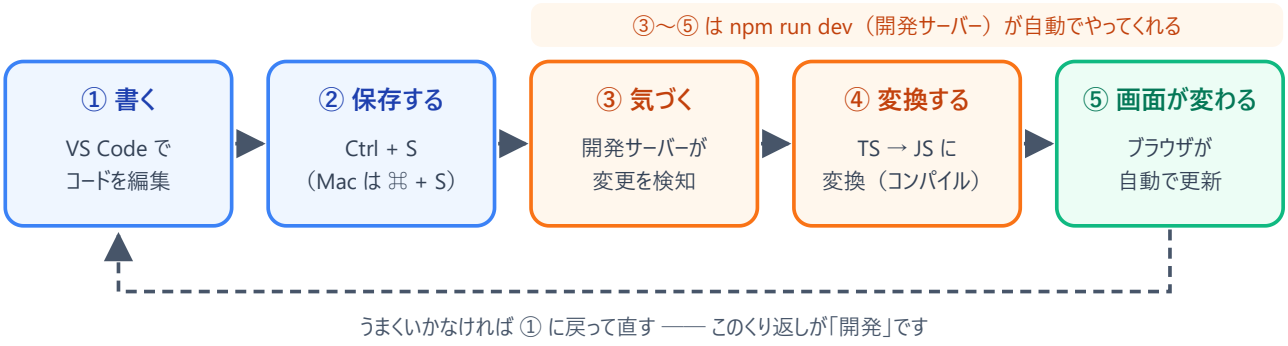


図0-7: コードを書いてから画面に出るまで。自分でやるのは①と②だけで、③〜⑤は開発サーバーが1秒前後で自動的に行います

> **ヒント:** この自動更新の仕組みを、Next.js では **Fast Refresh（ファスト・リフレッシュ）** と呼びます。保存しても画面が変わらないときは、①ファイルを保存し忘れていないか（VS Code のタブの名前の横に ● が付いていたら未保存）、② `npm run dev` を動かしているターミナルにエラーが出ていないか、の2つを確かめましょう。

> **つまずきポイント:** `npm run dev` を、プロジェクトのフォルダ**ではない場所**で実行してしまう。`npm run dev` は「今いるフォルダの `package.json` に書かれた `dev` の命令を実行する」コマンドなので、`package.json` が無い場所では何もできません。

NG: `package.json` が無いフォルダで実行した

```
PS C:\Users\taro> npm run dev
npm error code ENOENT
npm error syscall open
npm error path C:\Users\taro\package.json
npm error errno -4058
npm error enoent Could not read package.json: Error: ENOENT: no such file or directory,
open 'C:\Users\taro\package.json'
npm error enoent This is related to npm not being able to find a file.
npm error enoent
npm error A complete log of this run can be found in: C:\Users\taro\AppData\Local\npm-
cache\_logs\2026-10-15T11_29_14_015Z-debug-0.log
```

OK: プロジェクトのフォルダ（`package.json` がある場所）に移動してから実行する

```
# 第6章で作るプロジェクトのフォルダに移動する
cd book-management
# フォルダの中身を表示して、package.json があることを確かめる
ls
# 開発サーバーを起動する
npm run dev
```

- `ENOENT` は "Error NO ENTry" の略で、「そのファイル（ここでは `package.json`）が見つからない」という意味です。`npm error path` の行に、npm が探しに行った場所が書かれています。

サーバーが止まっているとどうなる？（つながらない体験）

> **▼ このコードがやること（先に日本語で）:** 誰も待ち受けていない窓口（3999 番ポート）に、わざとリクエストを送ってみます。サーバーが動いていないと「つながらない」というエラーになることを、自分の目で確かめるためのプログラムです。

```
// サーバーが待ち受けていない 3999 番の窓口に、リクエストを送ってみる
try {
  await fetch("http://localhost:3999/");
  console.log("つながりました");
} catch (error) {
  // つながらなかったときは、ここに来る
  console.log("つながりませんでした");
  if (error instanceof Error) {
    console.log(`エラーの内容: ${error.message}`);
    // 詳しい原因（cause）の中にある「エラーコード」を取り出す
    const cause = error.cause as { code: string };
    console.log(`原因のコード: ${cause.code}`);
  }
}
```

▼ 実行するコマンド

```
node refused.ts
```

▼ 実行結果

```
つながりませんでした
エラーの内容: fetch failed
原因のコード: ECONNREFUSED
```

▼ コードを1つずつ分解して解説

解説1: fetch でリクエストを送る

```
await fetch("http://localhost:3999/");
```

- `fetch(URL)` は「その URL にリクエストを送る」命令です。ブラウザのアドレスバーに URL を打つのとほぼ同じことを、プログラムから行います。
- `await`（アウェイト）は「返事が来るまで待つ」という意味です。通信には時間がかかるので、待つ必要があります（詳しくは第7章で学びます）。

解説2: try ... catch で失敗を受け止める

```
try {
  await fetch("http://localhost:3999/");
  console.log("つながりました");
} catch (error) {
  console.log("つながりませんでした");
}
```

（説明のため、`catch` の中身を一部だけ抜き出しています）

- `try { }` の中でエラーが起きると、プログラムを止めずに `catch { }` の中へジャンプします。いわば「失敗したときの受け皿」です。
- 今回は 3999 番で誰も待っていないので、`console.log("つながりました")` は実行されず、`catch` の中が実行されました。

解説3: エラーの中身を取り出す

```
if (error instanceof Error) {
  console.log(`エラーの内容: ${error.message}`);
  const cause = error.cause as { code: string };
  console.log(`原因のコード: ${cause.code}`);
}
```

- `error instanceof Error` は「受け取ったものが、本当にエラーの形をしているか」の確認です。TypeScript では、確認してからでないと `.message`（エラーの説明文）を読ませてもらえません。
- `error.cause` には「詳しい原因」が入っています。`as { code: string }` は「`code` という文字列を持っているものとして扱ってね」という TypeScript への申告です。
- `ECONNREFUSED` は "Error CONNection REFUSED"（接続を拒否された）の略で、「**その住所・その部屋番号には誰もいなかった**」という意味です。

> **つまずきポイント:** `npm run dev` を動かしていたターミナルを閉じたり、`Ctrl + C` で止めたりしたあとに `http://localhost:3000` を開く（または再読み込みする）。開発サーバーが止まっているので、上のプログラムと同じ「誰もいない部屋をノックした」状態になります。Chrome では「このサイトにアクセスできません」「localhost で接続が拒否されました。」「ERR_CONNECTION_REFUSED」と表示されます。
>
> - **NG:** 開発中に `npm run dev` のターミナルを閉じる → ブラウザで開いてもつながらない
> - **OK:** 開発中は `npm run dev` のターミナルを**開いたまま**にしておき、ほかのコマンドを打ちたいときは**新しいターミナルをもう1つ開く**。サーバーを止めたいときだけ `Ctrl + C` を押す

0.7「変数・関数・型」のひと言ミニ辞典

第2章以降で詳しく説明しますが、ここで先取りで意味だけ頭に入れておきましょう。

用語	ひと言で言うと	一行サンプル
変数（variable）	値に名前を付けて入れておく箱	`const name = "太郎";`
関数（function）	処理に名前を付けてまとめたもの。呼ぶと動く	`function greet() { ... }`
引数（ひきすう、argument）	関数を呼ぶときに渡す材料	`greet("太郎")` の `"太郎"`
戻り値（return value）	関数が処理結果として返す値	`return name;` の `name`
型（type）	「ここには文字列だけ入れてね」というルール	`name: string`
配列（array）	値の並び（リスト）	`[1, 2, 3]`
オブジェクト（object）	「キーと値」の組をまとめたもの	`{ name: "太郎", age: 20 }`

▼ 一行ずつの実行結果のイメージ:

```
// const = 「変更できない変数（定数）」を作る宣言キーワード / name = 変数名（自分で付ける箱の名前） / = は「右の値を左に入れる」代入演算子 / "太郎" はダブルクォートで囲まれた文字列（string型） / 末尾の ; は「文の終わり」を示すセミコロン
const name = "太郎";
// console = ブラウザやNode.jsが用意している「コンソール出力用オブジェクト」 / . は「オブジェクトの中の機能を呼ぶ」アクセス演算子 / log は「文字列などをコンソールに表示する関数」 / (name) で先ほど作った変数 name を引数として渡している / ; で文を終わる
console.log(name);

// ▼ 実行結果
// 太郎
```

変数と関数は、次のような「箱」と「機械」をイメージすると分かりやすくなります。

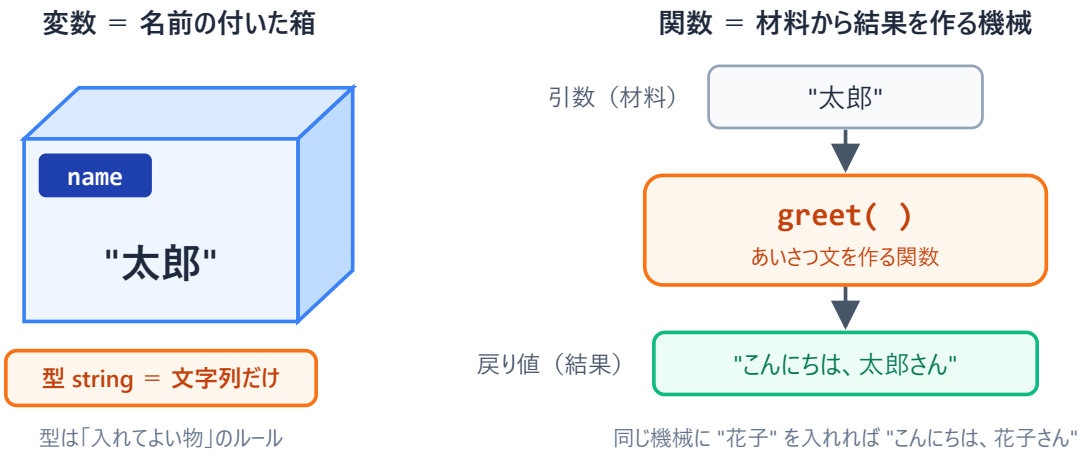


図0-8: 変数は名前の付いた箱、型はその箱に入れてよい物のルール、関数は材料（引数）を入れると結果（戻り値）が出てくる機械です

変数・関数・型をまとめて使ってみる

> **▼ このコードがやること（先に日本語で）**：「本」のデータの形（型）を決めて、本のデータを変数に入れ、そのデータから紹介文を作る関数を呼び出します。最後に、本を2冊並べた配列の冊数を表示します。上の表に出てきた用語（変数・関数・引数・戻り値・型・配列・オブジェクト）が全部登場します。

```
// 「本」のデータの形（型）を決める。Book という名前を付ける
type Book = {
  title: string;
  author: string;
  rating: number;
};

// 型 Book のルールに従って、本のデータを1冊ぶん作る
const book: Book = {
  title: "リーダブルコード",
  author: "Dustin Boswell",
  rating: 5,
};

// 本を受け取り、紹介文（文字列）を作って返す関数
function describe(b: Book): string {
  return `『${b.title}』（${b.author}）の評価は ★${b.rating} です`;
}

// 関数を呼び出して、返ってきた文字列を変数に入れる
const message = describe(book);
console.log(message);

// 本を2冊並べた配列（リスト）を作る
const shelf: Book[] = [
  book,
  { title: "達人プログラマー", author: "David Thomas", rating: 4 },
];

// 配列の長さ（何冊あるか）を表示する
console.log(`本棚には ${shelf.length} 冊あります`);
```

▼ 実行するコマンド（`book.ts` という名前で保存したフォルダで）

```
node book.ts
```

▼ 実行結果

```
『リーダブルコード』（Dustin Boswell）の評価は ★5 です
本棚には 2 冊あります
```

> **ヒント:** `.ts`（TypeScript）のファイルを `node` で直接動かせるのは、Node.js 22.18 以降などの新しいバージョンです。`ERR_UNKNOWN_FILE_EXTENSION` というエラーが出たら Node.js が古いので、第1章を参考に新しい LTS（長期サポート）版を入れてください。バージョンによっては `ExperimentalWarning`（試験的な機能です、という注意書き）が1行出ることがありますが、動作には問題ありません。また、このとき Node.js は型の部分を**読み飛ばして実行するだけ**で、型の間違いのチェックはしません。型のチェックは VS Code の赤い波線や、Next.js のビルド（第10章）が担当します。

▼ コードを1つずつ分解して解説

解説1: type で「データの形」を決める

```ts
type Book = {
  title: string;
  author: string;
  rating: number;
};
```

- `type 名前 = { ... }` で、「本のデータには title・author・rating の3つが必要で、それぞれの種類（型）はこれ」というルールに `Book` という名前を付けます。
- `string` は文字列、`number` は数値です。

解説2: 型に従ってデータを作る

```ts
const book: Book = {
  title: "リーダブルコード",
  author: "Dustin Boswell",
  rating: 5,
};
```

- `const book: Book` は「Book のルールに従った箱 `book` を作る」という意味です。
- `{ キー: 値, ... }` の形がオブジェクトです。もし `rating: "5"`（文字列）と書くと型のルールに合わないので、VS Code が赤い波線で教えてくれます。

解説3: 関数を作って呼び出す

```
function describe(b: Book): string {
  return `『${b.title}』（${b.author}）の評価は ★${b.rating} です`;
}

const message = describe(book);
```

- `function describe(b: Book): string` は、「Book 型の引数 `b` を受け取り、string 型の戻り値を返す関数 `describe` 」の定義です。
- `b.title` のように、`.`（ドット）でオブジェクトの中身を取り出します。
- `describe(book)` で関数を呼び出すと、`return` の値（紹介文）が戻ってきて `message` に入ります。

解説4: 配列と length

```
const shelf: Book[] = [
  book,
  { title: "達人プログラマー", author: "David Thomas", rating: 4 },
];

console.log(`本棚には ${shelf.length} 冊あります`);
```

- `Book[]` は「Book が並んだ配列」という型です。
- `shelf.length` で、配列に入っている要素の数（ここでは 2）が分かります。

> **つまずきポイント:** 大文字と小文字をまちがえる。プログラミング言語は、**`console` と `Console` をまったく別の名前**として扱います。

NG: 先頭を大文字で `Console` と書いた

```
Console.log("こんにちは");
```

```
ReferenceError: Console is not defined
```

（実際には、この行の前後にファイル名や `at ...` の行も表示されます。読み方は「10. この教材を進めるにあたってのアドバイス」で説明します）

OK: すべて小文字の `console` と書く

```
console.log("こんにちは");
```

```
こんにちは
```

- `ReferenceError: ○○ is not defined` は「○○という名前のものは定義されていない（見つからない）」という意味です。このエラーが出たら、まず**名前の打ち間違い（スペルミス・大文字小文字）**を疑いましょう。

> **つまずきポイント:** カッコや引用符を**全角**で打ってしまう。日本語入力のまま （ ） や “ ” （くるっと曲がった引用符）を打つと、コンピュータには「知らない記号」に見えます。Word やチャットからコードをコピーしたときにも、" が勝手に “ ” に変わっていることがあります。

NG①: 全角のカッコ

```
console.log（"こんにちは"）;
```

NG②: 曲がった引用符

```
console.log(“こんにちは”);
```

どちらも、実行すると次のエラーになります（ファイル名などの行は省略しています）。

```
SyntaxError: Invalid or unexpected token
```

OK: カッコも引用符も半角で打つ

```
console.log("こんにちは");
```

- `SyntaxError`（シンタックスエラー＝文法エラー）は「書き方のルールに反している」、`Invalid or unexpected token` は「おかしな文字・予想外の記号がある」という意味です。
- 文字列の**中身**（`"こんにちは"` の「こんにちは」の部分）は日本語で問題ありません。全角にしてはいけないのは、カッコ・引用符・セミコロンなど **コードの記号の部分** です。

ここまでが「読む前の超基礎」です。すべてを覚える必要はなく、「**分からない単語が出てきたらこの 0 章に戻ってくる**」という辞書のように使ってください。

---

1. この教材の目的と対象読者

目的

この教材は、以下のことを目的としています。

- **モダンなWeb開発の基礎**を、手を動かしながら体系的に学ぶ
- **TypeScript + React + Next.js + Supabase** という実務でも広く使われている技術スタック（technology stack：技術の組み合わせ）を習得する
- 一つのアプリを最初から最後まで作り切ることで、**開発の全体像**を理解する
- CRUD（クラッド。Create=作成、Read=読み取り、Update=更新、Delete=削除の頭文字で、データ操作の基本4つを指します）操作を通じて、**Webアプリ開発の基本パターン**を身につける

> **「CRUD」とは？** ほぼ全てのWebアプリは、データの「作成・読み取り・更新・削除」という4つの操作で成り立っています。SNSなら投稿の作成・表示・編集・削除、ECサイト（Electronic Commerce、ネット通販サイト）なら商品の登録・一覧表示・情報更新・削除です。この4つをマスターすれば、どんなWebアプリでも基本は作れるようになります。

書籍管理アプリでの CRUD を、画面での操作と、裏側で使われる「HTTP メソッド」「SQL」と並べると次のようになります。右側の2列は今は覚えなくて大丈夫です。「画面での1つの操作が、裏側では決まった言葉に置き換わって運ばれていく」というイメージだけ持っておきましょう。

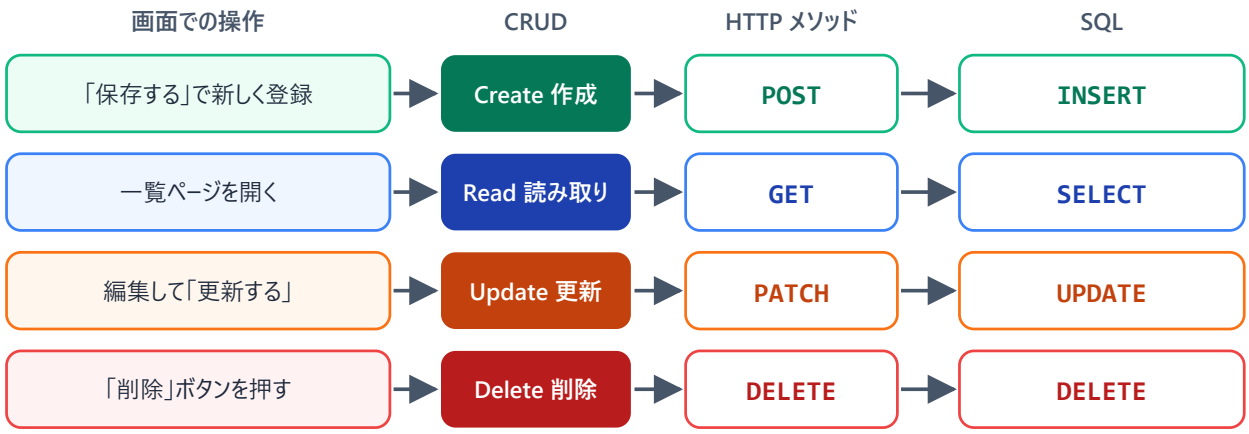


図0-9: CRUD の4つの操作と、画面での操作・HTTP メソッド・SQL の対応。左から右へ「画面 → 通信 → データベース」と置き換わって運ばれます

対象読者

この教材は、以下のような方を対象としています。

対象	説明
プログラミング初学者	HTML/CSSの基本は分かるが、本格的なWebアプリを作ったことがない方
他言語からの転向者	PythonやJavaなど他の言語の経験があり、Web開発を始めたい方
フロントエンド入門者	バックエンドの経験はあるが、React等のフロントエンド技術を学びたい方
学生・独学者	ポートフォリオに載せられる作品を作りたい方

> **安心してください！** 分からないことがあっても、各章で丁寧に説明します。エラーが出ても慌てず、一歩ずつ進めていきましょう。

---

2. Webアプリはどのように動いているのか

書籍管理アプリを作り始める前に、Webアプリがどのような仕組みで動いているのか、全体像を理解しておきましょう。ここで全てを覚える必要はありません。「ふーん、こういう仕組みなんだ」くらいの理解で十分です。

レストランに例えると

Webアプリの仕組みは、レストランに例えるとわかりやすくなります。

レストラン	Webアプリ	説明
メニュー・内装	**フロントエンド**（Frontend：ユーザーが直接見て触れる画面部分）	お客さんが見る・触れる部分。見た目の美しさや使いやすさが重要
厨房	**バックエンド**（Backend：サーバー側で動くプログラム。データの処理や保存を担当）	お客さんからは見えないが、注文を処理する裏方。データの処理を担当
食材倉庫	**データベース**（Database：データを永続的に保存する場所。電源を切ってもデータが消えない）	食材（データ）を保管する場所。必要なときに取り出せる
注文票	**API**（Application Programming Interface：アプリ同士がデータをやり取りするための窓口・ルール）	厨房と客席をつなぐ「注文の仕組み」。フロントエンドとバックエンドの通信手段
ウェイター	**HTTP通信**（HyperText Transfer Protocol：ブラウザとサーバー間でデータをやり取りする通信の決まりごと）	注文を厨房に届け、料理を客席に運ぶ人。データの運び役

表の内容を絵にすると、次のようになります。お客さん（ブラウザ）は、厨房（サーバー）にも倉庫（データベース）にも直接は入りません。必ずウェイター（HTTP 通信）を通して注文し、料理を受け取ります。

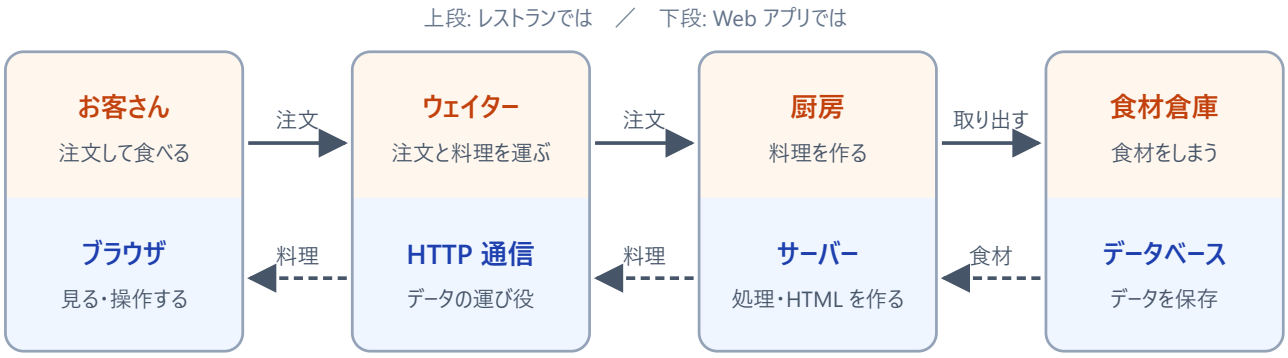


図0-10: Web アプリをレストランにたとえた図。実線の矢印が注文（リクエスト）、点線の矢印が返ってくるもの（レスポンス）です

Webアプリの動作の流れ

例えば、あなたがブラウザ（Chrome や Edge など）で「Amazon」を開いたとき、裏側ではこんなことが起きています。

1. **ブラウザがURLを入力** → 「amazon.co.jp の情報をください」とサーバーにリクエスト（要求）を送る
2. **サーバーがリクエストを受け取る** → 必要なデータをデータベースから取得する
3. **サーバーがHTMLを生成** → 商品情報を含んだページのデータを作成する
4. **ブラウザがHTMLを受け取る** → 受け取ったデータをもとに画面を描画（レンダリング、rendering：画面に絵を描く処理）する
5. **ユーザーが操作する** → 例えば「カートに追加」ボタンを押すと、再びサーバーにリクエストが送られる

> **このチュートリアルでは：** フロントエンド（画面）は **React + Next.js** で作り、バックエンド（データの処理・保存）は **Supabase** というサービスに任せます。Supabase を使うことで、バックエンドのプログラムを自分で書く必要がなくなり、画面の開発に集中できます。

小さな Web サーバーを動かして「リクエスト → レスポンス」を体験する

> **▼ このコードがやること（先に日本語で）：** 自分の PC の中に、HTML を1行だけ返す小さな Web サーバー（厨房）を作ります。そして同じプログラムの中から、ブラウザの代わりに `fetch` で「`/about` のページをください」とリクエスト（注文）を送り、返ってきたレスポンス（料理）を表示します。最後にサーバーを止めて終わります。

```ts
// Node.js に最初から入っている「HTTP サーバーを作る道具」を読み込む
import { createServer } from "node:http";

// サーバー（お店の厨房）を作る。リクエストが届くたびに中の関数が呼ばれる
const server = createServer((req, res) => {
  // どんな注文（リクエスト）が来たかを記録する
  console.log(`[サーバー] 注文を受けました: ${req.method} ${req.url}`);

  // 「成功(200)」と「中身は HTML です」という情報を先に返す
  res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });

  // 料理（HTML）を渡して、このやり取りを終える
  res.end("<h1>ようこそ、本の店へ</h1>");
});

// 3000 番の窓口（ポート）で、リクエストの待ち受けを始める
server.listen(3000, async () => {
  console.log("[サーバー] http://localhost:3000 で待っています");

  // ブラウザの代わりに fetch でリクエストを送ってみる
  const response = await fetch("http://localhost:3000/about");
  console.log(`[ブラウザ役] ステータスコード: ${response.status}`);

  // 返ってきた本文（HTML）を文字列として受け取る
  const html = await response.text();
  console.log(`[ブラウザ役] 届いた HTML: ${html}`);

  // 実験が終わったので、サーバーを止める
  server.close();
});
```

▼ **実行するコマンド**（`server.ts` という名前で保存したフォルダで）

```
node server.ts
```

▼ **実行結果**

```
[サーバー] http://localhost:3000 で待っています
[サーバー] 注文を受けました: GET /about
[ブラウザ役] ステータスコード: 200
[ブラウザ役] 届いた HTML: <h1>ようこそ、本の店へ</h1>
```

▼ コードを1つずつ分解して解説

解説1: サーバーを作る道具を読み込む

```
import { createServer } from "node:http";
```

- `import { 名前 } from "場所"` は、ほかの場所にある道具を「借りてくる」書き方です。
- `node:http` は Node.js に最初から入っている「HTTP 通信の道具箱」で、その中から `createServer`（サーバーを作る関数）を借りています。

解説2: 注文が来たときの動きを決める

```
const server = createServer((req, res) => {
  console.log(`[サーバー] 注文を受けました: ${req.method} ${req.url}`);
  res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
  res.end("<h1>ようこそ、本の店へ</h1>");
});
```

（コメントを省いて抜き出しています）

- `(req, res) => { ... }` は「リクエストが届くたびに呼ばれる関数」です。`req`（request）が届いた注文、`res`（response）が返事を書き込む用紙です。
- `req.method` は HTTP メソッド（今回は `GET`）、`req.url` はパス（今回は `/about`）です。
- `res.writeHead(200, ...)` で「成功（200）」と「中身は HTML で、文字コードは UTF-8」という情報を書き、`res.end(...)` で HTML を渡して返事を完了します。

解説3: ポートで待ち受けて、自分で注文を送る

```
server.listen(3000, async () => {
  const response = await fetch("http://localhost:3000/about");
  console.log(`[ブラウザ役] ステータスコード: ${response.status}`);
  const html = await response.text();
  server.close();
});
```

（説明に必要な行だけを抜き出しています）

- `server.listen(3000, ...)` で、3000 番ポートでの待ち受けを始めます。準備ができると、後ろに書いた関数が呼ばれます。
- `fetch(...)` がブラウザ役です。`response.status` に**ステータスコード**（200＝成功）、`await response.text()` に本文（HTML）が入ります。
- `server.close()` でサーバーを止めます。これが無いと、プログラムは次の注文を待ち続けて終わりません。

> **ヒント（やってみよう）**：最後の `server.close();` の行の先頭に `//` を付けて（コメントにして）実行すると、サーバーが動いたままになります。その間にブラウザで `http://localhost:3000/hello` を開くと「ようこそ、本の店へ」と表示され、ターミナルには `[サーバー] 注文を受けました: GET /hello` と記録されます（ブラウザはタブのアイコン画像 `/favicon.ico` も自動で注文するので、その記録も出ることがあります）。止めるときはターミナルで `Ctrl + C` を押します。これが、本書で使う `npm run dev`（開発サーバー）の小さな模型です。

> **つまずきポイント**: すでに別のプログラムが 3000 番の部屋（ポート）を使っているときに、もう1つ 3000 番で待ち受けようとする。たとえば `npm run dev` を動かしたまま、別のターミナルで上の `server.ts` を実行するとこうなります。

NG: 3000 番がふさがっているのに `node server.ts` を実行した

```
node:events:487
      throw er; // Unhandled 'error' event
      ^

Error: listen EADDRINUSE: address already in use :::3000
（このあとに at で始まる行などが続きますが、省略します）
```

OK: 先に動いているほうを `Ctrl + C` で止めてから実行する（または、`server.listen(3000, ...)` と `fetch("http://localhost:3000/about")` の `3000` を、空いている `3001` などに書き換える）

- `EADDRINUSE` は "Error ADDRess IN USE"（その住所はもう使われている）の略です。1つの部屋には1つのプログラムしか入れない、というポートのルールに引っかかりました。
- Next.js の開発サーバーは、3000 番がふさがっていると自動で 3001 番などに切り替え、ターミナルに `⚠ Port 3000 is in use by process 9076, using available port 3001 instead.`（3000 番は使用中なので、空いている 3001 番を使います。数字は場合によって変わります）のような警告を出します。その場合はブラウザでも `http://localhost:3001` を開く必要があります。`npm run dev` の出力に表示される `Local:` の URL を確かめるクセをつけましょう。

クライアントとサーバー

Webの世界では、「お願いする側」と「応える側」の2つの役割があります。

- **クライアント**（Client：サービスを利用する側）＝ あなたのブラウザ。「このページを見せて」「このデータを保存して」とお願いする側
- **サーバー**（Server：サービスを提供する側）＝ インターネット上のコンピュータ。お願いに応えてデータを返す側

この「クライアントがリクエストを送り、サーバーがレスポンス（応答）を返す」というやり取りが、Webアプリの基本です。

本書のアプリでは、この関係がもう一段だけ増えます。**Next.js のサーバー**が「厨房」として HTML を作り、その材料（データ）を **Supabase** という「倉庫」から取ってきます。どのコードがどこで動くのかを図にすると、次のようになります。

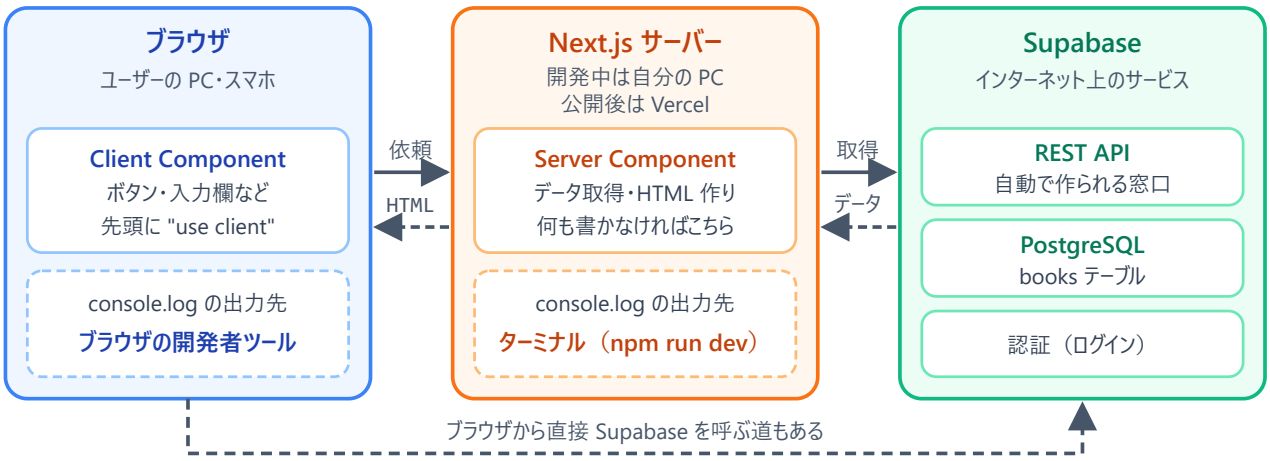


図0-11: どのコードがどこで動くか。Next.js のコードは「ブラウザで動く部品」と「サーバーで動く部品」に分かれ、console.log の出力先もそれぞれ違います

> **用語: 開発者ツール（DevTools、デブツールズ）** … ブラウザに付いている「中身をのぞく道具」です。Chrome や Edge では `F12` キー（Mac は `⌘ + Option + I`）で開き、「Console（コンソール）」タブに、ブラウザで動いたコードの `console.log` が表示されます。

> **つまずきポイント:** 「`console.log` を書いたのに、どこにも出てこない」——Next.js では、そのコードが**サーバーで動くか、ブラウザで動くか**によって、`console.log` の出力先が変わります。探す場所をまちがえているだけ、ということがとても多いです（第4章で実際に試します。ここでは読むだけで OK です）。

サーバーで動く例: `"use client"` が無いファイル（Server Component）

```
// app/page.tsx … "use client" が無いので Server Component（サーバーで動く）
export default function Page() {
  // この出力は「npm run dev を動かしているターミナル」に出る
  console.log("ページを作りました");
  return <p>こんにちは</p>;
}
```

ブラウザで動く例: 先頭に `"use client"` があるファイル（Client Component）

```
"use client";

// "use client" を先頭に書いたので Client Component（ブラウザで動く）
export default function LikeButton() {
  function handleClick() {
    // この出力は「ブラウザの開発者ツールの Console」に出る
    console.log("ボタンが押されました");
  }
  return <button onClick={handleClick}>いいね</button>;
}
```

- **NG:** サーバーで動くコードの出力を、ブラウザの開発者ツールだけで探し続ける（本番環境では、そこには出てきません）
- **OK:** サーバーで動くコード → **ターミナル** を見る。ブラウザで動くコード → **開発者ツールの Console** を見る。
- 開発中は、サーバー側の出力がブラウザの Console にも「Server」という印つきで表示されることがあります。それでも「本来の出力先はターミナル」と覚えておくと混乱しません。

---

3. 完成イメージ

このチュートリアルで作成する「書籍管理アプリ」は、以下のような画面構成になっています。

ホーム画面（書籍一覧）


書籍管理アプリ
＋ 新規登録
タイトルや著者名で検索...
リーダブルコード
読了
著者: Dustin Boswell
編集
削除
プロを目指す人のためのTypeScript入門
読書中
著者: 鈴木 僚太
編集
削除
達人プログラマー
読みたい
著者: David Thomas
編集
削除

書籍登録・編集画面



画面のつながり（画面遷移）

完成したアプリでは、4種類の画面を行き来します。ブラウザのアドレスバーに表示される URL（パス）にも注目してください。**URL のパスが変わると、表示される画面が変わる**——これが第4章で学ぶ **ルーティング** の考え方です。

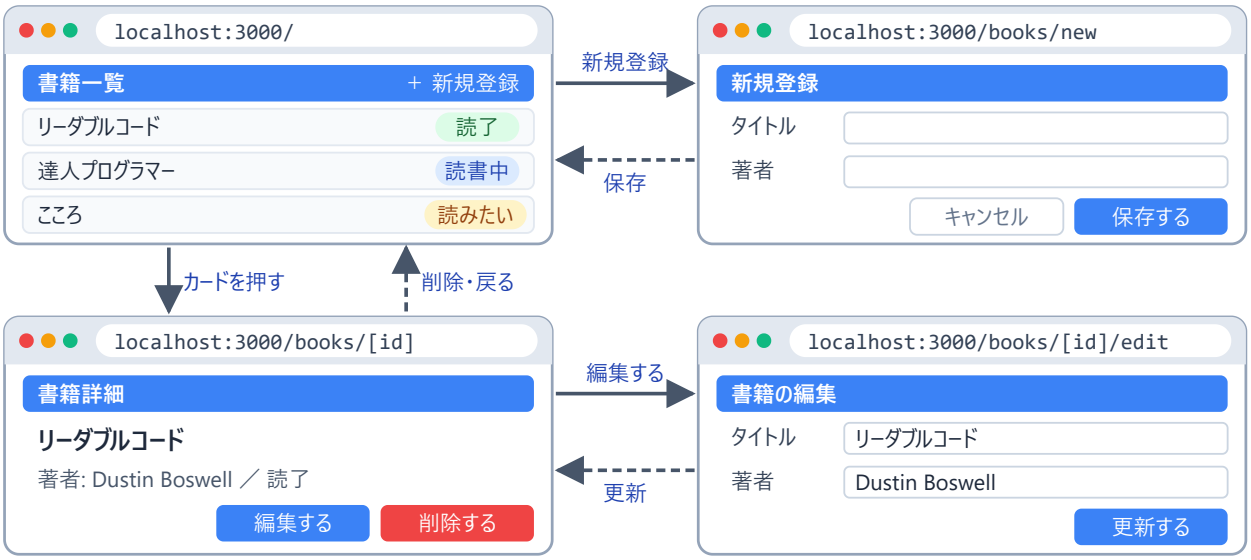


図0-12: 完成アプリの4つの画面と、画面どうしのつながり（画面イメージ）。アドレスバーの URL が、画面ごとに変わることに注目してください

> **用語:** 図の `/books/[id]` の `[id]` は、「ここには本ごとの番号（ID）が入る」という意味の目印です。実際には `/books/123` のように、その本の ID が入った URL になります（本書のアプリの ID は、もっと長い英数字の文字列です）。

主な機能

- **一覧表示**: 登録した書籍をリスト形式で表示
- **新規登録**: タイトル・著者・評価・ステータス・メモを入力して書籍を登録
- **編集**: 登録済みの書籍情報を変更
- **削除**: 不要な書籍データを削除
- **検索**: タイトルや著者名で書籍を絞り込み

主な機能を「ミニチュア」で体験する

> **▼ このコードがやること（先に日本語で）**：書籍管理アプリの心臓部である「登録・一覧・更新・削除（CRUD）」を、データベースの代わりにただの配列を使って再現します。本物のアプリでは、この配列の部分が Supabase のデータベースに置き換わるだけで、やっていることの考え方は同じです。

```ts
// 1冊の本が持つ情報の形（型）
type Book = {
  id: number;
  title: string;
  status: string;
};

// データベースの代わりに、ただの配列を「本棚」として使う
let books: Book[] = [];
// 次に登録する本に付ける番号
let nextId = 1;

// C: Create（作成）… 新しい本を本棚に追加する
function createBook(title: string): void {
  books.push({ id: nextId, title: title, status: "want_to_read" });
  nextId = nextId + 1;
}

// R: Read（読み取り）… 本棚の中身を全部表示する
function listBooks(): void {
  for (const b of books) {
    console.log(`  ${b.id}: ${b.title} [${b.status}]`);
  }
}

// U: Update（更新）… 指定した番号の本の状態を書き換える
function updateStatus(id: number, status: string): void {
  const target = books.find((b) => b.id === id);
  if (target) {
    target.status = status;
  }
}

// D: Delete（削除）… 指定した番号「以外」の本だけを残す
function deleteBook(id: number): void {
  books = books.filter((b) => b.id !== id);
}

createBook("リーダブルコード");
createBook("達人プログラマー");
console.log("① 2冊を登録した直後");
listBooks();

updateStatus(1, "completed");
console.log("② 1番を読了(completed)にした後");
listBooks();

deleteBook(2);
```

```
console.log("③ 2番を削除した後");
listBooks();
```

▼ 実行するコマンド（`crud.ts` という名前で保存したフォルダで）

```
node crud.ts
```

▼ 実行結果

```
① 2冊を登録した直後
  1: リーダブルコード [want_to_read]
  2: 達人プログラマー [want_to_read]
② 1番を読了(completed)にした後
  1: リーダブルコード [completed]
  2: 達人プログラマー [want_to_read]
③ 2番を削除した後
  1: リーダブルコード [completed]
```

▼ コードを1つずつ分解して解説

解説1: 配列を「本棚」として使う

```
let books: Book[] = [];
let nextId = 1;
```

- `[]` は空っぽの配列です。ここに本を入れていきます。本物のアプリでは、これが Supabase のデータベースにある `books` テーブルにあたります。
- `let` は「あとで中身を入れ替えられる変数」です（`const` は入れ替え不可）。削除のときに配列ごと入れ替えるので、`let` にしています。
- `nextId` は、次に登録する本に付ける番号です。本物のデータベースでは、ID は自動で付けてくれます。

解説2: Create — push で追加する

```
function createBook(title: string): void {
  books.push({ id: nextId, title: title, status: "want_to_read" });
  nextId = nextId + 1;
}
```

- `books.push(...)` で、配列の最後に本を1冊追加します。登録したばかりの本の状態は `"want_to_read"`（読みたい）です。本書のアプリでも、読書ステータスは `want_to_read`（読みたい）・`reading`（読書中）・`completed`（読了）の3つの値を使います。
- 戻り値の型の `void`（ボイド）は、「この関数は何も返さない」という意味です。

解説3: Read — for...of で全部表示する

```
function listBooks(): void {
  for (const b of books) {
    console.log(`  ${b.id}: ${b.title} [${b.status}]`);
  }
}
```

- 本棚の本を1冊ずつ取り出して表示します。アプリでは、これが「一覧画面」にあたります。

解説4: Update — find で探して書き換える

```
const target = books.find((b) => b.id === id);
if (target) {
  target.status = status;
}
```

- `books.find(条件)` は、条件に合う最初の1冊を探します。`(b) => b.id === id` は「本 `b` の id が、指定された id と等しい」という条件を表す小さな関数（アロー関数）です。
- 見つからなかったときは `undefined`（何もない）が返るので、`if (target)` で「見つかったときだけ」書き換えます。

解説5: Delete — filter で「それ以外」を残す

```
books = books.filter((b) => b.id !== id);
```

- `filter(条件)` は、「条件に合うものだけを集めた新しい配列」を作ります。`!==` は「等しくない」なので、「指定した id **以外**の本」だけが残り、結果として指定した本が消えます。

> **ヒント:** `find` や `filter`、`=>` を使った書き方は第2章でじっくり練習します。今は「CRUD の4つの操作は、データの入れ物に対する **追加・表示・書き換え・削除** なんだ」とつかめれば十分です。本物のアプリでは、`books.push(...)` が Supabase の `insert`、`books.find(...)` が `select`、のように置き換わります。

---

4. 学習ロードマップ

以下のステップで、段階的にアプリを完成させていきます。焦らず、一つずつ進めていきましょう。「ロードマップ（roadmap）」は「道筋を示した地図」という意味の英単語です。



第0章: はじめに（この章）

全体概要・技術スタック・学習ロードマップ

↓



第1章: 開発環境のセットアップ

Node.js / VS Code / Git / ターミナルを準備する

↓



第2章: TypeScript の基礎

型の基本を学ぶ

↓



第3章: React の基礎

コンポーネント・State・イベントの考え方

↓



第4章: Next.js の基礎

App Router・Server / Client Components

↓



第5章: Supabase セットアップとデータベース設計

データベースを準備する

↓



第6章: プロジェクトセットアップ

書籍管理アプリの土台を作る

↓



第7章: 一覧表示と登録機能

CRUD の Read と Create

↓



第8章: 編集・削除・詳細表示

CRUD の Update と Delete、検索・並び替え

↓

第9章: スタイリング & UI ポリッシュ
見た目と使いやすさを仕上げる

↓

第10章: デプロイと総まとめ
アプリをインターネットに公開する

↓

完成！
書籍管理アプリの完成です。おめでとうございます！

↓

第11章: Claude Code × Next.js バイブコーディング（発展）
AI と一緒に、さらに機能を足していく

各章の目安学習時間は **30分〜1時間程度** です。全体を通して、**約1〜2週間** で完走できる構成になっています。

11 の章は、大きく4つのフェーズ（段階）に分けられます。どのフェーズで何ができるようになるのかを先に知っておくと、「今なぜこれを学んでいるのか」が分かりやすくなります。第1〜4章は少し地味に感じるかもしれませんが、第5章からのアプリ作りを支える「基礎体力」の期間です。

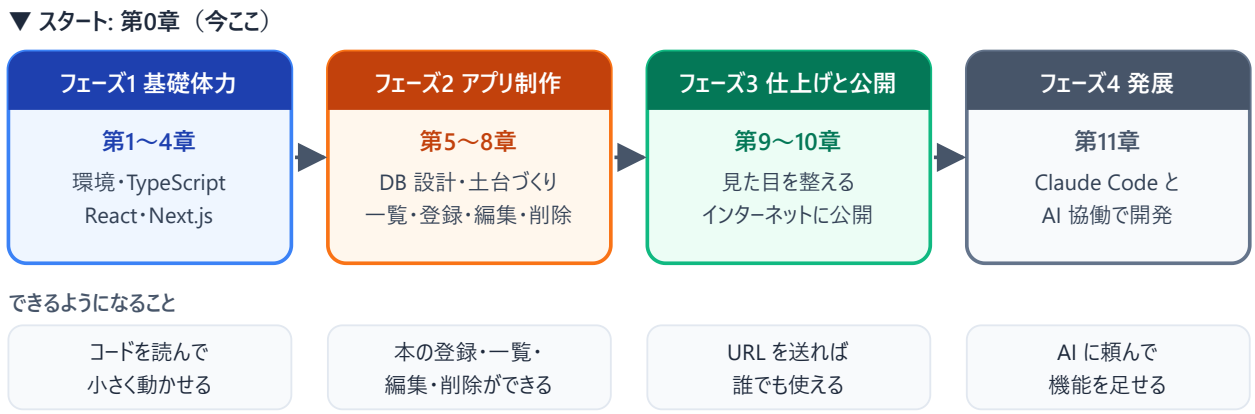


図0-13: 学習ロードマップを4つのフェーズに分けた図。下の段は、各フェーズを終えたときにできるようになることです

---

5. 使用技術スタック一覧

このチュートリアルで使用する技術の一覧と、それぞれの役割をまとめます。「スタック（stack）」は「積み重ね」という意味で、複数の技術を組み合わせて使うことを指します。

技術	バージョン目安	カテゴリ	役割
TypeScript	5.x	プログラミング言語	JavaScript（Webブラウザで動くプログラミング言語）に「型」（データの種類の指定）を追加した言語。コードの安全性と可読性を向上させる
React	19.x	UIライブラリ（User Interface Library：画面の部品を作るための道具箱）	ユーザーインターフェース（画面）を効率的に構築するためのライブラリ。Meta社（旧Facebook）が開発
Next.js	16.x	Webフレームワーク（Framework：アプリ開発の骨組みを提供するソフトウェア）	Reactベースのフレームワーク。ルーティング（URLに応じたページの切り替え）やサーバーサイド機能を提供する。Vercel社が開発
Supabase	-	BaaS（Backend as a Service：バックエンド機能をクラウドで提供するサービス）	データベース・認証・APIを提供するサービス。バックエンドを自前で構築する手間を省ける。オープンソース
Node.js	24（LTS）推奨・22 以上	ランタイム（Runtime：プログラムを実行する環境）	JavaScriptをブラウザの外（自分のPC上）で実行するための環境。開発ツールの動作に必要
npm	11.x（Node.js 24 に付属）	パッケージマネージャ（Package Manager：ライブラリの管理ツール）	他の開発者が作った便利なプログラム（パッケージ）をインストール・管理するツール
Git	2.x	バージョン管理システム（VCS：コードの変更履歴を記録する仕組み）	コードの変更履歴を記録・管理するツール。「いつ、誰が、何を変えたか」を追跡できる
VS Code	最新版	コードエディタ（IDE：統合開発環境に近い高機能エディタ）	コードを書くためのエディタ。Microsoft社が開発。豊富な拡張機能が利用可能

> **「バージョン」って何？** ソフトウェアは改良されるたびに番号が振り直されます。「5.x」と書いてあれば「メジャー番号が 5 の系列ならどの細かい版でも OK」という意味です。`x` の部分は「何でもいい」を表すワイルドカードです。表記は SemVer（セマンティックバージョニング、Semantic Versioning）と呼ばれ、`メジャー.マイナー.パッチ` の3階層が標準です。

技術スタックの役割分担を図で見る

表だけだとイメージしにくいので、**家づくり**にたとえて図にしてみます。Next.js が家の骨組み、React がその中にはめ込む部品、TypeScript は部品を作るときの材料の規格、Supabase は家の外にある倉庫、Vercel は家を建てる土地です。Node.js や VS Code は、家を建てるための大工道具にあたります。

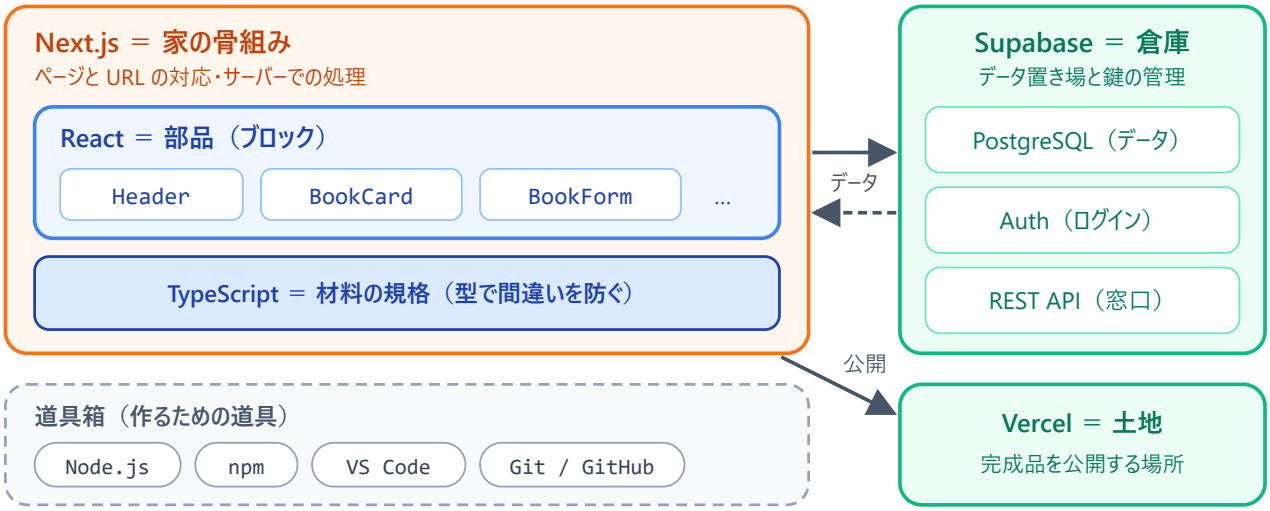


図0-14: 技術スタックの役割分担。Next.js の骨組みの中に React の部品をはめ込み、すべてを TypeScript で書きます。データは Supabase に置き、完成品は Vercel で公開します

各技術の概要

TypeScript — 「型」のある JavaScript

TypeScriptは、JavaScriptに**静的型付け**（Static Typing：コードを実行する前に、変数や関数に入るデータの種類をチェックする仕組み）の機能を追加した言語です。

「この変数には文字列しか入らない」「この関数は数値を返す」といったルールをコードに書けるため、間違いを事前に防ぐことができます。

> **身近な例え：** 型は「ラベル付きの箱」のようなものです。「みかん専用」と書いてある箱にりんごを入れようとすると、入れる前に「それはみかんじゃないよ！」と教えてくれます。プログラムでも同じように、間違ったデータを入れようとすると、コードを書いている段階でエラーが表示されます。

```
// ================================================================================
// 比較サンプル: 同じ「足し算」を JS と TS で書いた場合
// --------------------------------------------------------------------------------
// この例は「TypeScriptがいかに早い段階でバグを教えてくれるか」を示すための比較。
// ================================================================================
// で始まる行は「コメント」と呼ばれ、プログラムとしては実行されない。
// 上の行のように
// 自分や読み手向けの説明文として自由に書ける。

// ---------- JavaScript の場合（型なし） ----------
// function は「関数を定義する」キーワード（予約語）
// add は関数名。自分で好きに名付けられる（小文字始まりが慣習）
// (a, b) は引数リスト。a と b の2つの値を受け取る、と宣言している
// 型注釈が無いので a も b も「どんな種類の値でもOK」（= 動的型付け）
function add(a, b) {
  // return は「関数の結果を呼び出し元へ返す」キーワード
  // a + b: + 演算子は「両方が数値なら算術加算」「片方でも文字列なら文字列連結」になる
  return a + b;
  // 末尾の ; （セミコロン）は「この文はここで終わり」を示す
}

// (1) "1" は文字列リテラル（ダブルクォートで囲むと文字列扱い）、2 は数値リテラル
//     JavaScriptは「文字列 + 数値」を「文字列の連結」として処理する。
//     よって "1" + 2 → "12" という文字列が返ってくる。
// add 関数を呼び出している。( ) の中が引数。複数あればカンマ , で区切る
add("1", 2);
// ▼ 実行結果（変数に入れて console.log すると）
// "12"   ← 数字の12ではなく文字列の "12"。気づきにくいバグ！


// ---------- TypeScript の場合（型あり） ----------
// 引数の後ろに「: 型名」を書くと、その引数は指定した型しか受け付けなくなる
// a: number → a は number（数値）型のみ
// b: number → b も number 型のみ
// ) の後ろの「: number」は「この関数の戻り値の型」を指定している（戻り値も数値）
function add(a: number, b: number): number {
  // a も b も数値だけなので、ここでは必ず数値の加算になる
  return a + b;
}

// (2) "1" は文字列なので、a: number に渡すと型の不一致になりエラー
add("1", 2);
// ▼ コンパイル時のエラー（VS Codeでは赤い波線で即表示。「コンパイル」=実行前の変換処理）
// Argument of type 'string' is not assignable to parameter of type 'number'.
//   （'string' 型の引数は 'number' 型のパラメータには代入できません）
//
// → 実行する前にミスに気づける！  これが TypeScript の最大の価値。
```

React — コンポーネントで画面を構築

Reactは、画面を**コンポーネント**（Component：UIの部品。ボタン、カード、フォームなど、画面の構成要素一つひとつ）という小さな部品に分割して構築するライブラリ（Library：特定の機能を提供するプログラムの集まり）です。

> **身近な例え：** レゴブロックを想像してください。小さなブロック（コンポーネント）を組み合わせて、家や車（ページ全体）を作ります。一度作ったブロックは、別の作品でも再利用できます。Reactも同じで、「書籍カード」というコンポーネントを一度作れば、一覧画面で何回でも使い回せます。

Next.js — Reactをさらに便利にするフレームワーク

Next.jsは、Reactだけでは面倒な**ページ遷移の管理**（ルーティング：URLに応じて表示するページを切り替える仕組み）や**サーバー側での処理**（SSR：Server Side Rendering、サーバー上でHTMLを生成してからブラウザに送る方式）を簡単に実現できるフレームワーク（Framework：アプリ開発の骨組みや決まりごとを提供するソフトウェア）です。

> **Reactだけだと何が困るの？** Reactは画面の「部品」を作ることに特化したライブラリなので、「URLを変えたらページを切り替える」「検索エンジンに見つけてもらう」「サーバーでデータを取得する」といった機能は自分で用意する必要があります。Next.jsはこれらを最初から備えており、ファイルを配置するだけでページが作れる仕組みが特徴的です。

Supabase — バックエンドを手軽に構築

Supabaseは、データベース（データの保管庫）やユーザー認証（ログイン機能。認証＝authentication、本人確認）などの**バックエンド機能**をクラウド（インターネット上のサーバー）で提供するサービスです。自分でサーバーを構築する必要がなく、設定するだけですぐにデータの保存・取得ができます。

> **自分でバックエンドを作るとどうなるの？** サーバーの設定、APIの設計、データベースの構築、セキュリティ対策…と、やることが一気に増えます。Supabaseを使えば、これらを全てクラウド上で数クリックで用意でき、フロントエンド（画面）の開発に集中できます。学習中は特にこれが重要です。

---

6. なぜこの技術スタックを選んだのか

技術にはたくさんの選択肢があります。ここでは、今回の技術スタックを選んだ理由を、他の選択肢と比較しながら説明します。

UIライブラリの比較: React vs Vue vs Svelte

観点	React	Vue	Svelte
学習コスト	やや高め	低め	低め
求人・需要	非常に多い	多い	少なめ
エコシステム	非常に豊富	豊富	成長中
コミュニティ	世界最大級	大きい（特にアジア圏）	成長中
TypeScriptサポート	良好	良好	良好
大規模開発への適性	高い	高い	発展途上

> 「**エコシステム**」とは？ ある技術を中心にした関連ツールや拡張機能、書籍、コミュニティ全体のこと。豊かなエコシステム=「困った時に解決策が見つけやすい」という意味でもあります。

Reactを選んだ理由: 最も広く使われているUIライブラリであり、就職・転職活動でも求められることが多いため、学ぶ価値が非常に高いです。エコシステムが充実しているので、困ったときに情報を見つけやすいのも大きなメリットです。

フレームワークの比較: Next.js vs Vite vs Remix

観点	Next.js	Vite + React	Remix
セットアップの簡単さ	簡単	簡単	やや手間がかかる
サーバーサイド機能	充実（SSR/SSG/ISR）	なし（SPA中心）	充実
ファイルベースルーティング	あり	なし（要追加設定）	あり
デプロイのしやすさ	非常に簡単（Vercel）	簡単	やや手間がかかる
学習リソースの豊富さ	非常に多い	多い	少なめ
企業での採用実績	非常に多い	多い	増加中

> **SSR/SSG/ISR/SPA の用語整理:** - SSR (Server Side Rendering)：サーバーでHTMLを作ってからブラウザに送る方式。- SSG (Static Site Generation)：ビルド時にHTMLを作っておく方式。表示が速い。- ISR (Incremental Static Regeneration)：SSG をベースに一定間隔でHTMLを再生成する方式。- SPA (Single Page Application)：1枚のHTMLでブラウザ側がページを切り替える方式。

Next.jsを選んだ理由: ファイルを作るだけでページが追加できる仕組みや、Vercelへのワンクリックデプロイなど、初心者にとって「余計なことを考えずに済む」設計が魅力です。実務での採用例も非常に多く、学んだことがそのまま活かせます。

言語の比較: TypeScript vs JavaScript

観点	TypeScript	JavaScript
型安全性	あり（コンパイル時にエラー検出）	なし（実行時にエラー発覚）
コードの可読性	高い（型が仕様書の役割を果たす）	普通
学習コスト	やや高め（型の概念の学習が必要）	低め
開発効率	高い（エディタの補完が強力）	普通
業界のトレンド	主流になりつつある	依然として広く利用
エラーの発見しやすさ	書いている途中で気づける	実行してから気づく

TypeScriptを選んだ理由: 最初は少し学ぶことが増えますが、型があることでエディタが強力に補助してくれるため、結果的に**初心者にもやさしい**言語です。「何を渡せばいいか分からない」という場面が大幅に減ります。

バックエンドの比較: Supabase vs Firebase vs 自前バックエンド

観点	Supabase	Firebase	自前バックエンド
データベースの種類	PostgreSQL（リレーショナルDB）	Firestore（NoSQL）	自由に選択可能
SQL の学習	できる	できない	できる
無料枠	十分（個人開発に最適）	十分	サーバー費用が必要
セットアップの手間	非常に簡単	簡単	大変（サーバー構築が必要）
学習コスト	低い	低い	高い（API設計等が必要）
オープンソース	はい	いいえ	-
他のサービスへの移行	しやすい（標準SQL）	しにくい（独自仕様）	-

> **「リレーショナルDB / NoSQL」とは？** - リレーショナルDB（Relational Database, RDB）：表（テーブル）と表の関係でデータを管理する方式。SQLで操作する。PostgreSQL、MySQL など。 - NoSQL（Not Only SQL）：表形式に縛られないデータベースの総称。Firestore は「ドキュメント型」のNoSQL。

Supabaseを選んだ理由: PostgreSQL（業界標準のデータベース）を使っているため、ここで学んだSQL知識は他の環境でもそのまま活かせます。また、オープンソースであり、ダッシュボードが直感的で使いやすく、無料枠も個人学習には十分です。

---

7. 前提知識

このチュートリアルを始めるにあたって、以下の知識があると学習がスムーズに進みます。

必須の前提知識

- **HTML の基本**: タグ（`<div>`, `<p>`, `<a>` など）の役割が分かること。タグは `<開始タグ>中身</終了タグ>` の形で書く決まりがあります。
- **CSS の基本**: 文字色や背景色の変え方、簡単なレイアウトの作り方が分かること。`color: red;` のように「プロパティ名: 値;」の形で書きます。
- **プログラミングの基本概念**: 変数、条件分岐（if文）、繰り返し（for文）、関数がどういうものか分かること

あると望ましい知識（なくても大丈夫です）

- **JavaScript の基本**: 変数宣言（`const` / `let`）、アロー関数（`() => { ... }` の形の短い関数の書き方）、配列操作（`map`, `filter`：配列を加工・抽出する関数）
- **コマンドライン操作**: ターミナル（コマンドプロンプト）で `cd`（change directory：移動先のフォルダを切り替える）や `ls`（list：今いるフォルダの中身一覧。Windows では `dir`）が使えること
- **Git の基本**: `git add`（変更を「コミット予定リスト」に登録）、`git commit`（コミット予定をスナップショットとして記録）の意味が分かること

> **前提知識に不安がある方へ:** 心配しなくても大丈夫です。必要な知識はその都度、補足説明を入れています。「まったくの未経験」でなければ、十分についてこれる内容です。分からない部分があったら、飛ばさずにゆっくり読み返してみてください。

---

8. この教材で学べること

チュートリアルを完走すると、以下のスキルが身につきます。一つずつチェックを付けながら進めていきましょう！（Markdown の `- [ ]` は「未完了のチェックボックス」を表す書き方です）

開発環境

- [ ] Node.js と npm のインストールと基本操作
- [ ] VS Code のセットアップと便利な拡張機能の導入
- [ ] Git によるバージョン管理の基本

TypeScript

- [ ] 基本的な型（`string`：文字列, `number`：数値, `boolean`：真偽値=true/false）の使い方

- [ ] 型定義（`type` / `interface`：複雑な型に名前を付ける2つの方法）の作成
- [ ] 関数の引数と戻り値に型を付ける方法

React

- [ ] コンポーネントの概念と作成方法
- [ ] JSX（JavaScript XML：JS/TS の中にHTMLっぽい記法を直接書ける構文）の書き方
- [ ] `useState`（ステートを保持するためのフック関数）を使った状態管理
- [ ] `useEffect`（副作用を扱うためのフック関数）を使った副作用の処理
- [ ] Props（プロパティ：親コンポーネントから子コンポーネントへ渡すデータ）によるデータの受け渡し
- [ ] イベントハンドリング（クリック・入力など：ユーザー操作を受け取って反応する処理）
- [ ] フォームの作成とバリデーション（validation：入力値の妥当性チェック）

Next.js

- [ ] プロジェクトの作成と構成の理解
- [ ] App Router によるファイルベースルーティング（ファイル/フォルダ構成がそのままURLになる仕組み）
- [ ] Server Components と Client Components の違い
- [ ] レイアウトの共通化

Supabase

- [ ] プロジェクトの作成とテーブル設計
- [ ] データの作成（INSERT：SQLでデータを新規追加する命令）
- [ ] データの読み取り（SELECT：SQLでデータを取り出す命令）
- [ ] データの更新（UPDATE：SQLでデータを書き換える命令）
- [ ] データの削除（DELETE：SQLでデータを消す命令）
- [ ] フィルタ・検索機能の実装

総合スキル

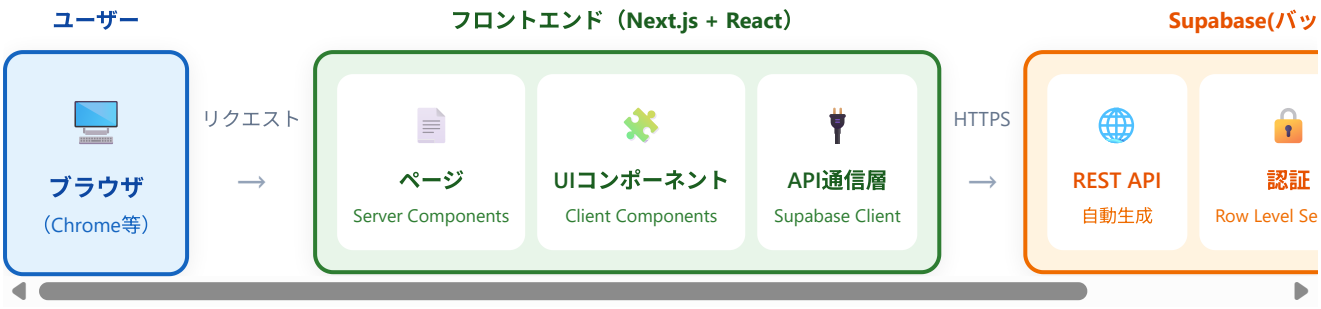
- [ ] モダンなWebアプリの設計パターンの理解
- [ ] CRUD操作の一連の流れの実装
- [ ] アプリケーションのデプロイ（公開）

---

9. アプリのアーキテクチャ（全体構成図）

最終的に完成するアプリの全体像を、以下の図で確認しましょう。今は全てを理解する必要はありません。チュートリアルを進めるうちに、それぞれの役割が自然と分かるようになります。「アーキテクチャ（architecture）」は「建築物の構造／設

計」という意味で、ソフトウェアでは「全体の構成」を指します。



books テーブル

`id` `title` `author` `rating` `status` `memo` `created_at` `updated_at`

「books テーブル」の各カラム（列）の意味: - `id` … レコード（行）を一意に識別するための番号。主キー（Primary Key）。 - `title` … 書籍タイトル（文字列）。 - `author` … 著者名（文字列）。 - `rating` … 評価（1〜5 の整数。まだ評価していなければ空＝null）。 - `status` … 読書状態。`want_to_read` （読みたい）／ `reading` （読書中）／ `completed` （読了）のいずれか。 - `memo` … 自由記述メモ（文字列、長め）。 - `created_at` … レコードが作成された日時。自動で入る。 - `updated_at` … レコードが最後に更新された日時。自動で入る。

ほかにも出版社・出版日・表紙画像の URL などの列があります。テーブルの設計は第5章でくわしく行います。

データの流れ（具体例: 書籍を新規登録する場合）

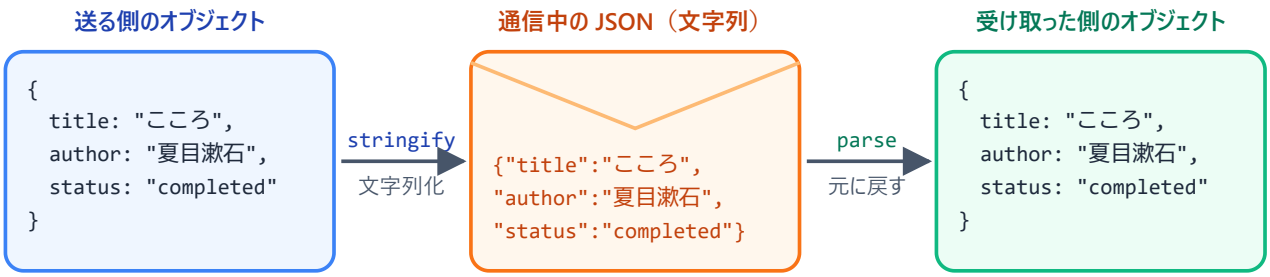
👤 ユーザー	🧩 React UI	🔦 Supabase Client	🌐 REST API	🗄️ PostgreSQL

1	**ユーザー → React UI**	フォームに書籍情報を入力
2	**ユーザー → React UI**	「保存する」ボタンをクリック
3	**React UI → Supabase Client**	`insert({ title, author, ... })`
4	**Supabase Client → REST API**	`POST /rest/v1/books`
5	**REST API → PostgreSQL**	`INSERT INTO books ...`
6	**PostgreSQL ← REST API**	登録完了（新しいレコード）
7	**REST API ← Supabase Client**	レスポンス（登録データ）
8	**Supabase Client ← React UI**	登録結果を受け取る
9	**React UI → ユーザー**	「登録しました！」と表示
10	**React UI → React UI**	書籍一覧を再取得して画面を更新

> **流れの読み方:** ステップ1〜5は「ユーザーから DB へ向かう登録リクエスト」、ステップ6〜10は「DB から画面へ戻ってくる応答」と覚えると分かりやすいです。`POST` はHTTPメソッドの一種で「新規作成」を意味します。`INSERT INTO books ...` はSQL（データベース操作言語）の構文で「books テーブルに行を追加」する命令です。

データは「JSON」という形で運ばれる

上の流れのステップ4〜7で、書籍のデータはインターネットを通って運ばれます。このとき、プログラムの中のデータ（オブジェクト）はそのままでは運べないので、**JSON（ジェイソン、JavaScript Object Notation）** という「決まった書き方の文字列」に変換されてから送られます。手紙を封筒に入れて送り、受け取った人が封筒から出して読む、というイメージです。



Supabase とのやり取りでは、この変換をライブラリが自動で行います

図0-15: JSON の旅。送る側で JSON.stringify により文字列に変換し、受け取った側で JSON.parse により元のオブジェクトに戻します

JSON への変換と、元に戻す変換を試す

> **▼ このコードがやること（先に日本語で）**：本のデータ（オブジェクト）を `JSON.stringify` で JSON の文字列に変換して表示し、その文字列を `JSON.parse` で元のオブジェクトに戻します。`typeof` を使って「本当に文字列になったか・オブジェクトに戻ったか」も確かめます。

```
// プログラムの中で扱う「本」のデータ（オブジェクト）
const book = {
  title: "こころ",
  author: "夏目漱石",
  status: "completed",
};

// 通信で送れるように、オブジェクトを JSON という「ただの文字列」に変換する
const json = JSON.stringify(book);
console.log(json);
// typeof は「この値の種類は何か」を調べる仕組み
console.log(typeof json);

// 受け取った側で、JSON の文字列を元のオブジェクトに戻す
const received = JSON.parse(json);
console.log(received.author);
console.log(typeof received);
```

▼ 実行するコマンド（`json.ts` という名前で保存したフォルダで）

```
node json.ts
```

▼ 実行結果

```
{"title":"こころ","author":"夏目漱石","status":"completed"}
string
夏目漱石
object
```

▼ コードを1つずつ分解して解説

解説1: JSON.stringify で文字列にする

```
const json = JSON.stringify(book);
console.log(json);
console.log(typeof json);
```

- `JSON.stringify(オブジェクト)` は、オブジェクトを JSON の文字列に変換します（stringify ＝ 文字列にする）。
- 結果を見ると、キー（`title` など）も `"` で囲まれ、全体が1行の文字の並びになっています。これが JSON の書き方のルールです。
- `typeof json` が `string`（文字列）なので、本当に「ただの文字列」になったことが分かります。

解説2: JSON.parse で元に戻す

```
const received = JSON.parse(json);
console.log(received.author);
console.log(typeof received);
```

- `JSON.parse(文字列)` は、JSON の文字列をオブジェクトに戻します（parse ＝ 解析する）。
- オブジェクトに戻ったので、`received.author` のように `.` で中身を取り出せます。`typeof` も `object` になりました。

> **ヒント:** Supabase とやり取りするときは、この変換を Supabase のライブラリ（ `@supabase/supabase-js` ）が自動でやってくれるので、自分で `JSON.stringify` を書くことはほとんどありません。それでも「裏ではこういう形で運ばれている」と知っておくと、ブラウザの開発者ツールで通信の中身をのぞいたときに読めるようになります。

まとめ

この章では、以下のことを確認しました。

- このチュートリアルで作る **書籍管理アプリ** の完成イメージ
- **学習ロードマップ** と各章の概要

- 使用する **技術スタック** とそれぞれの役割
- なぜこの技術スタックを **選んだのか** の理由
- チュートリアルを始めるための **前提知識**
- この教材で **学べること** の全体像
- アプリの **アーキテクチャ**（全体構成）
- ブラウザ・サーバー・URL・ポート・ターミナルといった **基本用語** と、小さなプログラムを `node` で動かす方法
- エラーが出たときに **どこを読めばいいか**（`〇〇Error:` の行、ファイル名と行番号）と、よくあるつまずきの直し方

> **準備はできましたか？** 次の章では、実際に開発環境を構築していきます。Node.jsやVS Codeのインストールから始めましょう！

---

10. この教材を進めるにあたってのアドバイス

プログラミング学習を効果的に進めるためのアドバイスをまとめました。

エラーが出ても慌てない

プログラミングでは、エラーは**日常的に起こるもの**です。プロのエンジニアでも、毎日のようにエラーに遭遇しています。エラーメッセージは「ここが間違っているよ」と教えてくれるヒントなので、むしろ**ありがたい存在**です。

> **エラーメッセージの読み方のコツ：** エラーメッセージは英語で表示されることがほとんどですが、慌てず以下の手順で対処しましょう。1. エラーメッセージの中から **`〇〇Error:` で始まる行**を探して読む（最も重要な情報が書かれています。Python などのエラーでは最後の行に出ますが、Node.js / JavaScript のエラーでは上のほうに出て、最後の行はバージョン番号であることが多いので注意） 2. メッセージをそのまま**Google で検索**する（同じエラーに遭遇した人の解決策が見つかります） 3. このチュートリアルの**トラブルシューティング**（trouble shooting：問題解決）セクションを確認する

エラーメッセージを実際に読んでみる

> **▼ このコードがやること（先に日本語で）**：本の値段と冊数から合計金額を計算するつもりのプログラムですが、変数名を1か所打ち間違えています（ `price` を `prise` と書いた）。わざとエラーを出して、エラーメッセージのどこに何が書いてあるかを確かめます。

```
// 本1冊の値段と、買う冊数
const price = 1500;
const count = 3;

// 値段 × 冊数 で合計金額を計算する（ここに打ち間違いがある）
const total = prise * count;

console.log(`合計は ${total} 円です`);
```

▼ **実行するコマンド**（`total.js` という名前で保存したフォルダで）

```
node total.js
```

▼ **実行結果**（フォルダの場所の部分は、人によって異なります）

```
C:\Users\taro\study\total.js:6
const total = prise * count;
              ^

ReferenceError: prise is not defined
    at Object.<anonymous> (C:\Users\taro\study\total.js:6:15)
    at Module._compile (node:internal/modules/cjs/loader:1829:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1969:10)
    at Module.load (node:internal/modules/cjs/loader:1552:32)
    at Module._load (node:internal/modules/cjs/loader:1354:12)
    at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v25.9.0
```

長くて英語ばかりなので身構えてしまいますが、部分ごとに色分けすると、読むべき所はほんの少しだと分かります。

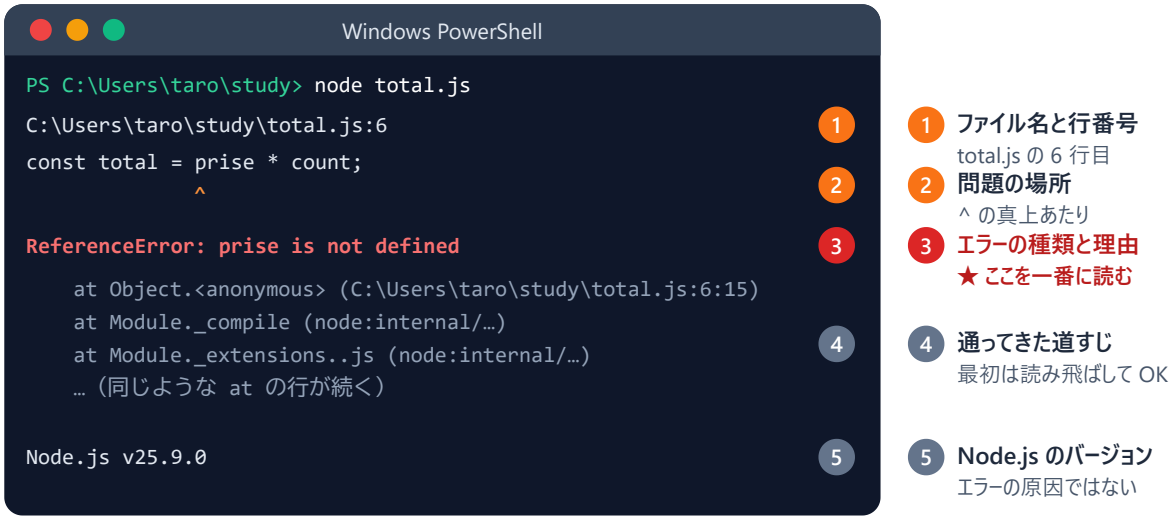


図0-16: エラーメッセージの読み方（画面イメージ）。まず③の「〇〇Error:」の行を読み、①②で場所を確かめます。④⑤は最初は気にしなくて大丈夫です

▼ エラーメッセージを1つずつ分解して解説

解説1: どのファイルの何行目か（①）

```
C:\Users\taro\study\total.js:6
```

- 「`total.js` の **6 行目** でエラーが起きた」という意味です。VS Code で 6 行目を見に行きましょう（`Ctrl + G` を押して行番号を入れるとジャンプできます）。

解説2: 問題の場所（②）

```
const total = prise * count;
              ^
```

- 6 行目の中身がそのまま表示され、その下の `^`（キャレット）が問題の場所を指しています。ここでは `prise` の先頭を指しています。

解説3: エラーの種類と理由（③ 一番大事）

```
ReferenceError: prise is not defined
```

- `:` の左が **エラーの種類**、右が **理由** です。`ReferenceError`（参照エラー）は「その名前のものが見つからない」、`prise is not defined` は「prise は定義されていない」という意味です。
- 2 行目で作ったのは `price` なので、`prise` は打ち間違いだと分かります。

解説4: at で始まる行と、最後の行（④⑤）

- `at ...` の行は、エラーが起きるまでにプログラムが通ってきた道すじ（**スタックトレース**と呼びます）です。`(…\total.js:6:15)` の `6:15` は「6 行目の 15 文字目」という意味です。自分のファイル名が出ている行以外は、最初は読み飛ばして構いません。
- 最後の `Node.js v25.9.0` は、エラーの内容ではなく、使っている Node.js のバージョンです。

直したコード（`prise` → `price`）と、その実行結果です。

```
// 本1冊の値段と、買う冊数
const price = 1500;
const count = 3;

// 値段 × 冊数 で合計金額を計算する（打ち間違いを直した）
const total = price * count;

console.log(`合計は ${total} 円です`);
```

```
合計は 4500 円です
```

> **つまずきポイント:** エラーメッセージの**最後の行だけ**を見て、「`Node.js v25.9.0` って何？ 何が悪いのか分からない」と迷ってしまう。Node.js のエラーでは、大事な `〇〇Error:` の行は上のほうにあり、最後の行はバージョン表示です。
>
> - NG: 最後の行だけを読む → `Node.js v25.9.0`（原因は書かれていない）
> - OK: `Error:` を含む行を探して読む → `ReferenceError: prise is not defined`（原因が書かれている）
>
> なお、日本語（全角文字）を含む行では、`^` の位置が少し左右にずれて見えることがあります。そのときは `total.js:6:15` のような「行番号:文字目」の数字を頼りにしましょう。

写経（しゃきょう）から始めよう

最初はコードの意味が完全に理解できなくても、**まずはサンプルコードをそのまま書き写してみましょう**（これを「写経」と呼びます）。手を動かすことで、自然と「こういうパターンなんだな」と体が覚えていきます。

写経をしたら、そこで終わりにせず、**少しだけ変えて動かしてみる**のがおすすめです。わざと壊してエラーを読み、直してまた動かす——このサイクルを回すほど、「分かる」が増えていきます。この章の `hello.js` や `crud.ts` も、文字や数字を書き換えて遊んでみてください。

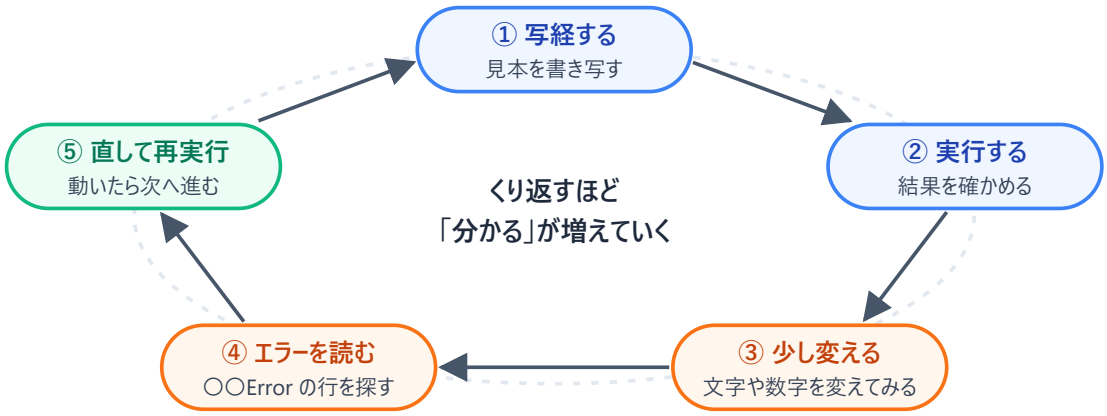


図0-17: 学習のサイクル。写経して動かしたら、少し変えて、エラーを読んで、直す。この1周ごとに理解が深まります

分からない箇所は飛ばしてOK

全てを完璧に理解してから次に進む必要はありません。「今は分からないけど、後で分かるようになるかも」と思って、**とりあえず先に進む**のも大切な学習戦略です。後の章で実際にコードを書いたときに、「ああ、あのとき説明されていたのはこういうことか！」と腑に落ちることがよくあります。

各章の目安時間

章	テーマ	目安時間
第0章	はじめに（この章）	30〜45分（練習問題を含む）
第1章	開発環境の構築	30〜60分
第2章	TypeScript入門	45〜90分
第3章	React入門	60〜90分
第4章	Next.js入門	45〜90分
第5章	Supabase設定・DB設計	30〜60分
第6章	プロジェクト作成	30〜45分
第7章	一覧・登録機能	60〜90分
第8章	編集・削除・検索	60〜90分
第9章	スタイリング・UI改善	45〜60分
第10章	デプロイ	30〜45分
第11章	Claude Code でバイブコーディング（発展・任意）	60〜120分

全体を通して**約1〜2週間**（1日1〜2時間のペース）で完走できる構成です。もちろん、もっとゆっくりでもまったく問題ありません。

11. よくある質問（FAQ）

FAQ は Frequently Asked Questions（よく聞かれる質問）の略です。

Q: プログラミング完全未経験でも大丈夫ですか？

A: HTML/CSSの基本（タグの書き方や、文字色の変え方など）と、プログラミングの基本概念（変数、if文、for文、関数とは何か）の知識があれば大丈夫です。これらに不安がある場合は、先にProgateやドットインストールなどの入門サービスで基礎を学んでおくとスムーズです。

Q: WindowsとMac、どちらでも進められますか？

A: はい、どちらでも問題なく進められます。第1章の環境構築の手順は、Windows/Mac/Linuxそれぞれに対応しています。

Q: このチュートリアルで作ったアプリは、ポートフォリオ（portfolio：作品集）に使えますか？

A: もちろん使えます。ただし、チュートリアル通りに作っただけではなく、**自分なりのアレンジ**（機能の追加、デザインの変更、別のテーマへの応用など）を加えると、より魅力的なポートフォリオになります。

Q: 有料のサービスは使いますか？お金はかかりますか？

A: このチュートリアルで使用するサービス（Supabase, Vercel, GitHub）はすべて**無料枠**で利用できます。クレジットカードの登録も不要です（Supabaseの無料プランで十分です）。

Q: エラーが解決できないとき、どうすればいいですか？

A: 以下の順序で試してみてください。1. このチュートリアルのトラブルシューティングセクションを確認 2. エラーメッセージをそのままGoogle検索 3. ChatGPTやClaude（AIアシスタント）にエラーメッセージを貼り付けて質問 4. Stack Overflow（プログラマー向けQ&Aサイト）で検索 5. GitHubのIssues（プロジェクトの質問掲示板）で類似の問題を検索

> **さあ、始めましょう！** 次の章では、開発に必要なツール（Node.js、VS Code、Git）をインストールしていきます。

12. 用語集（Glossary）— 困ったらここへ戻ってくる

このチュートリアルを読んでいて分からない単語に出会ったら、まずこの用語集を見てください。**完璧に覚える必要はありません。**「あ、こういう意味だったな」と思い出せれば十分です。Glossary は「用語集」を表す英単語です。

開発環境・ツール系

用語	読み	意味
Node.js	ノードジェイエス	ブラウザ外でJavaScript/TypeScriptを動かす実行環境。`node コマンド` で使える
npm	エヌピーエム	Node Package Manager。便利なプログラム部品をダウンロード・管理するツール
パッケージ	ぱっけーじ	他の人が作った再利用可能なプログラムのまとまり
ライブラリ	らいぶらり	特定機能を提供するパッケージ。例：React は UI ライブラリ
フレームワーク	ふれーむわーく	アプリ全体の骨組みを提供する大きめの仕組み。例：Next.js
VS Code	ぶいえすこーど	Microsoft製の高機能コードエディタ。無料
ターミナル	たーみなる	キーボードから命令を打って操作する画面（黒い画面）
コマンドプロンプト	こまんどぷろんぷと	Windowsの古めのターミナル
PowerShell	ぱわーしぇる	WindowsのターミナルNo.2。本書ではこちらを使う
Bash	バッシュ	macOS/Linux で標準的なターミナルのシェル
シェル	しぇる	ターミナルでコマンドを解釈する本体
Git	ぎっと	コードのバージョン（変更履歴）管理ツール
GitHub	ぎっとはぶ	Gitで管理したコードを保管する世界最大のサービス
リポジトリ（repo）	りぽじとり	Git/GitHubでコードを保管する単位。プロジェクト1つにつき1つ作るのが普通
コミット	こみっと	Gitで「この時点のスナップショットを記録」する操作
プッシュ	ぷっしゅ	ローカルのコミットをGitHubに送る操作
プル	ぷる	GitHubの最新コミットを自分のPCに取り込む操作

言語・型系

用語	読み	意味
HTML	エイチティーエムエル	文書の構造を書く言語（Hyper Text Markup Language）
CSS	シーエスエス	見た目を指定する言語（Cascading Style Sheets）
JavaScript（JS）	じゃばすくりぷと	ブラウザで動くスクリプト言語
TypeScript（TS）	たいぷすくりぷと	JavaScriptに「型」を追加した言語
型（Type）	かた	「数値」「文字列」など、データの種類のラベル
静的型付け	せいてきかたづけ	コード実行前に型をチェックする方式（TS, Java など）
動的型付け	どうてきかたづけ	コード実行時に型が決まる方式（JS, Python など）
変数	へんすう	値を入れる箱。`const`, `let` で宣言する
定数	ていすう	一度入れたら変えられない変数。`const` で宣言
関数	かんすう	処理をまとめて名前を付けたもの。`function` で定義
引数	ひきすう	関数に渡す入力値
戻り値	もどりち	関数が返す値。`return` で指定
配列	はいれつ	値の並び。`[1, 2, 3]`
オブジェクト	おぶじぇくと	キーと値のペアの集合。`{ name: "太郎" }`
アロー関数	あろーかんすう	`() => { ... }` の形式の関数
インターフェース	いんたーふぇーす	TS で型をまとめて定義する仕組み
ジェネリクス	じぇねりくす	型をパラメータとして受け取る仕組み（`<T>`）
async/await	エイシンク/アウェイト	非同期処理を書きやすくするキーワード
Promise	ぷろみす	非同期処理の結果を表すオブジェクト

Web/通信系

用語	読み	意味
HTTP	エイチティーティーピー	ブラウザとサーバーの通信規約
HTTPS	エイチティーティーピーエス	HTTPの暗号化版
URL	ユーアールエル	Webページの住所（例：`https://example.com/about`）
リクエスト	りくえすと	クライアントからサーバーへの要求
レスポンス	れすぽんす	サーバーからクライアントへの応答
GET	げっと	データの取得に使うHTTPメソッド
POST	ぽすと	データの新規登録に使うHTTPメソッド
PUT/PATCH	ぷっと/ぱっち	データの更新に使うHTTPメソッド
DELETE	でりーと	データの削除に使うHTTPメソッド
ステータスコード	すてーたすこーど	レスポンスの状態を示す3桁の数字。`200=成功` `404=見つからない` `500=サーバーエラー`
API	エーピーアイ	プログラム同士が通信するための窓口
REST API	れすとえーぴーあい	URLとHTTPメソッドで操作するAPIの設計スタイル
JSON	じぇいそん	データをやり取りする形式。`{"name":"太郎"}` のような形
ポート	ぽーと	同じPC内のどの窓口かを示す番号（例：3000）
localhost	ろーかるほすと	「自分自身のPC」を表す住所
DNS	ディーエヌエス	ドメイン名（`example.com`）をIPアドレスに変換する「インターネットの住所録」
IPアドレス	アイピーアドレス	ネットにつながった機械の番地。`192.0.2.10` のような数字の並び
開発者ツール	かいはつしゃつーる	ブラウザ内蔵の調査道具（DevTools）。`F12` で開き、Consoleタブに `console.log` が出る
Fast Refresh	ふぁすとりふれっしゅ	保存するとブラウザの画面が自動で更新される Next.js の仕組み

React/Next.js 系

用語	読み	意味
コンポーネント	こんぽーねんと	画面を構成する再利用可能な部品
JSX	じぇいえすえっくす	JS/TSの中にHTMLっぽい記法を埋め込む書き方
Props	ぷろっぷす	コンポーネントの外から渡すデータ
State	すてーと	コンポーネントの内部で持つ変化するデータ
useState	ゆーすすてーと	Reactで状態を持たせるためのフック関数
useEffect	ゆーすえふぇくと	Reactで副作用（データ取得など）を行うためのフック
フック	ふっく	`use〜` で始まるReactの関数群（Hooks）
App Router	あっぷるーたー	Next.jsの新しいページ管理方式（フォルダ＝URL）
Server Components	さーばーこんぽーねんと	サーバー側で実行されるコンポーネント（デフォルト）
Client Components	くらいあんとこんぽーねんと	ブラウザ側で実行されるコンポーネント（ `"use client"` を書く）
SSR	エスエスアール	Server Side Rendering。サーバーでHTMLを生成
CSR	シーエスアール	Client Side Rendering。ブラウザ上でHTMLを生成
SSG	エスエスジー	Static Site Generation。ビルド時にHTMLを生成
ISR	アイエスアール	Incremental Static Regeneration。一定間隔でHTMLを再生成
ハイドレーション	はいどれーしょん	サーバーで作られたHTMLにJSの動きを付ける処理

データベース/Supabase 系

用語	読み	意味
データベース（DB）	でーたべーす	データを永続的に保存する場所
RDB	アールディービー	リレーショナルデータベース。表形式
PostgreSQL	ぽすとぐれすきゅーえる	オープンソースのRDB。Supabaseが内蔵
SQL	エスキューエル	データベースを操作する言語
テーブル	てーぶる	データを保存する表
カラム（列）	からむ	テーブルの縦の項目（フィールド）
レコード（行）	れこーど	テーブルの横の1件分のデータ
主キー（Primary Key）	しゅきー	レコードを一意に識別するカラム。普通は `id`
外部キー（Foreign Key）	がいぶきー	他のテーブルの主キーを参照するカラム
CRUD	くらっど	Create / Read / Update / Delete の頭文字。データ操作の基本4つ
Supabase	すーぱべーす	オープンソースのBaaS。PostgreSQL＋認証＋API
RLS	アールエルエス	Row Level Security。行単位の権限制御
環境変数	かんきょうへんすう	設定値を `.env` ファイルなどに分離して書く仕組み

その他

用語	読み	意味
デプロイ	でぷろい	作ったアプリをサーバーに配置して公開する作業
Vercel	ばーせる	Next.jsを作った会社のホスティングサービス。本書でデプロイ先に使う
ホスティング	ほすてぃんぐ	作ったアプリをサーバーで動かして公開してくれるサービス
ビルド	びるど	開発用のコードを本番用に変換・最適化する処理
バンドル	ばんどる	多数のJSファイルを1〜数個にまとめる処理
ESLint	いーえすりんと	JS/TSのコードをチェックして悪い書き方を警告するツール
Prettier	ぷりてぃあー	コードの見た目（インデントなど）を自動整形するツール
スタックトレース	すたっくとれーす	エラーの下に並ぶ `at ...` の行。エラーまでにプログラムが通った道すじ
ENOENT	イー・ノー・エント	「ファイルが見つからない」ことを表すエラーコード
ECONNREFUSED	イー・コン・リフューズド	「接続を拒否された（その窓口に誰もいない）」ことを表すエラーコード
EADDRINUSE	イー・アドレス・イン・ユース	「そのポートはもう使われている」ことを表すエラーコード

---

13. 練習問題（解答・解説つき）

この章の内容を、7つの問題で確かめましょう。まず自分で考えてから、後ろの「解答と解説」を読んでください。プログラムを動かす問題（問題5・問題7）は、第1章で Node.js を入れたあとに挑戦すれば大丈夫です。

問題

問題1: URL を部品に分ける

`http://localhost:3000/?status=reading` という URL について、(1) スキーム、(2) ホスト名、(3) ポート番号、(4) パス、(5) クエリ文字列 をそれぞれ答えてください。

問題2: 拡張子から中身を当てる

次の4つのファイルには、それぞれ何が書かれているでしょうか。

- `page.tsx`
- `package.json`
- `globals.css`
- `README.md`

問題3: CRUD に分類する

書籍管理アプリの次の操作は、CRUD のどれにあたりますか。裏側で使われる HTTP メソッドも答えてください。

1. 一覧ページを開く
2. 本の状態を「読書中」から「読了」に変える
3. 新しい本を登録する
4. もう要らない本を消す

問題4: 担当はだれ？

次の仕事を主に担当するのは、TypeScript・React・Next.js・Supabase のどれでしょう。

1. 書籍のデータを保存しておく
2. `/books/new` という URL に、登録ページを対応させる
3. 「書籍カード」という画面の部品を作って、何度も使い回す
4. 数値を入れるべき所に文字列を入れたら、実行する前に教えてくれる

問題5: エラーを読んで直す

次のプログラムは本のタイトルを表示したいのですが、3か所まちがっています。実行すると最初に出るエラーは何でしょうか。また、3か所をどう直せばよいでしょうか。

```
// 本のタイトルを表示したい（でも 3 か所まちがっている）
const title = "リーダブルコード";
Console.log ("タイトル: " + titel) ;
```

問題6: ターミナルの操作

本や記事に、次のように書いてありました。

```
$ npm run dev
```

1. 実際にターミナルに打つ文字は、どこからどこまでですか。
2. 起動したあと、ブラウザで開く URL は何ですか（3000 番が空いている場合）。
3. 開発サーバーを止めるには、どうしますか。
4. 止めたあとにブラウザで再読み込みすると、どうなりますか。

問題7: 自分でプログラムを書く

変数 `myName`（名前）と `bookCount`（本の冊数。例: 3）を作り、テンプレートリテラルを使って「花子さんの本棚には 3 冊の本があります」と表示するプログラム `shelf.ts` を書いてください。余裕があれば、冊数を1増やした文も表示して

みましょう。

解答と解説

問題1の解答

(1) `http:`（http）、(2) `localhost`、(3) `3000`、(4) `/`、(5) `?status=reading`

解説: 0.3節の `url.ts` と同じように、プログラムに確かめさせることもできます。

```
// 問題1の URL を部品に分ける
const url = new URL("http://localhost:3000/?status=reading");
console.log(url.protocol);
console.log(url.hostname);
console.log(url.port);
console.log(url.pathname);
console.log(url.search);
console.log(url.searchParams.get("status"));
```

```
http:
localhost
3000
/
?status=reading
reading
```

- `localhost:3000` は、0.6節で見た「自分の PC の 3000 番の部屋」です。ここでは http の既定値（80）ではないので、ポート番号がきちんと `3000` と表示されます。
- パスの `/` は「サイトのいちばん上（トップページ）」を表します。本書のアプリでは、書籍の一覧ページが `/` です。
- `?status=reading` のようなクエリ文字列は、「読書中の本だけ」のような絞り込みの条件を URL に載せるときによく使われる形です。

問題2の解答

- `page.tsx` … TypeScript + JSX で書かれた画面（コンポーネント）のファイル。Next.js では、`page.tsx` という名前のファイルが1つのページになります。
- `package.json` … プロジェクトの設定（使う部品の一覧や、`npm run dev` で動く命令など）を書いた JSON 形式のファイル。
- `globals.css` … アプリ全体の見た目（色・余白など）を指定する CSS のファイル。
- `README.md` … プロジェクトの説明を書いた Markdown 形式の文書。

解説: 拡張子を見れば、中身の種類が分かります（0.2節）。`.tsx` と `.ts` の違いは、JSX（HTML に似た書き方）が入っているかどうかです。

問題3の解答

1. Read（読み取り）・GET
2. Update（更新）・PATCH
3. Create（作成）・POST
4. Delete（削除）・DELETE

解説: 図0-9 の対応表のとおりです。2 は「新しく作る」のではなく「すでにある本の一部を書き換える」ので Update です。Supabase を使うと、HTTP メソッドの使い分けはライブラリが自動でやってくれますが、「どの操作が CRUD のどれか」を意識できると、第7章・第8章のコードがぐっと読みやすくなります。

問題4の解答

1. Supabase
2. Next.js
3. React
4. TypeScript

解説: 図0-14 の家づくりのたとえで言うと、1 は倉庫（Supabase）、2 は家の骨組み＝どの部屋（URL）に何を置くか（Next.js）、3 は部品（React）、4 は材料の規格チェック（TypeScript）です。

問題5の解答

最初に出るエラーはこれです（ファイル名などの行は省略しています）。

```
SyntaxError: Invalid or unexpected token
```

直すべき3か所は、① 全角のカッコ `（ ）` → 半角の `( )`、② `Console` → `console`、③ `titel` → `title` です。

```
// 本のタイトルを表示する（3 か所を直した版）
const title = "リーダブルコード";
console.log("タイトル: " + title);
```

```
タイトル: リーダブルコード
```

解説: エラーは**1つずつ順番に**報告されます。実際に1か所ずつ直しながら実行すると、エラーは次のように変わっていきました。

1. 何も直さずに実行 → `SyntaxError: Invalid or unexpected token`（全角のカッコ）
2. カッコを半角に直す → `ReferenceError: Console is not defined`
3. `console` に直す → `ReferenceError: titel is not defined`

4. `title` に直す → `タイトル: リーダブルコード` と表示される

- 文法のまちがい（`SyntaxError`）は、プログラムを動かす前の「読み込み」の段階で見つかるので、いちばん先に報告されます。名前のまちがい（`ReferenceError`）は、その行を実行したときに初めて見つかります。
- 「1つ直したのに、また別のエラーが出た！」は失敗ではなく、**前に進んでいる証拠**です。

問題6の解答

1. `npm run dev` だけを打ちます。`$` はプロンプト記号なので打ちません（0.5節）。
2. `http://localhost:3000` です。
3. `npm run dev` を動かしているターミナルで `Ctrl + C` を押します。
4. 開発サーバーが止まっているので、Chrome なら「このサイトにアクセスできません」「ERR_CONNECTION_REFUSED」と表示され、アプリは表示されません。

解説: 4 は 0.6節の `refused.ts` で体験した `ECONNREFUSED` と同じ状態です。開発中は `npm run dev` のターミナルを開いたままにしておきましょう。3000 番がふさがっていた場合は 3001 番などに切り替わるので、`npm run dev` の出力の `Local:` の行で URL を確かめます。

問題7の解答（例）

```
// 自分の名前と、本棚の冊数を変数に入れる
const myName: string = "花子";
const bookCount: number = 3;

// テンプレートリテラル（バッククォート）で文章に埋め込んで表示する
console.log(`${myName}さんの本棚には ${bookCount} 冊の本があります`);

// 1冊増えたことにして、もう一度表示する
console.log(`1冊買ったので ${bookCount + 1} 冊になりました`);
```

▼ 実行するコマンド

```
node shelf.ts
```

▼ 実行結果

```
花子さんの本棚には 3 冊の本があります
1冊買ったので 4 冊になりました
```

解説:

- バッククォート `` ` `` で囲んだ文字列の中に `${変数}` と書くと、変数の値が埋め込まれます。

- `${bookCount + 1}` のように、`${ }` の中には計算式も書けます。このとき `bookCount` そのものは 3 のまま変わらず、表示する値だけが 4 になる点に注目してください。
- `: string` や `: number` は書かなくても動きますが、書いておくと「ここには文字列」「ここには数値」というルールが見える化され、まちがった値を入れたときに VS Code が教えてくれます。

> **さあ、始めましょう！** 次の章では、開発に必要なツール（Node.js、VS Code、Git）をインストールしていきます。

次の章へ進む → 第1章: 開発環境の構築